

Course Overview

Course Overview

Hi, everyone. My name is Zoran Horvat, and welcome to my course on Collections and Generics in C#. I'm a principal consultant at Coding Helmet. In 20 years of programming, I have witnessed numerous designs where only a well-crafted data structure could help solve practical problems. You will find some of those marvelous designs in this course, in which you will walk the evolutionary path from using collections over encapsulating collections to isolate them from domain-related code and on to designing your own powerful unique collections that have no match in the .NET BCL. Some of the major topics that we will cover include applying collections that operate within time and the memory bounds, constructing reusable domain-related classes on top of collections, designing custom general-purpose collections, and using and constructing generic types and methods. By the end of this course, you will feel confident to use collections even in the most demanding design, and there will be no mystery in generics for you. Before beginning the course, you should be familiar with operating principles of BCL collections and the C# programming language. From here, you should feel comfortable diving into courses on advanced object-oriented design, LINQ, but also on functional design with C#. I hope you will join me on this journey with the Collections and Generics in C# course at Pluralsight.

Introducing Collections and Generics

Introduction

Hello, and welcome to the course on Collections and Generics in C#. My name is Zoran Horvat. In the next few hours, I will walk you through practical aspects of using collections and generics in software development. In this introductory module, you will learn about the division of collections and see a

demonstration why we need collections in the first place. Interestingly, the answer to this last question will not be so obvious. Before we begin, I wanted to make a note on technologies that will be employed. Everything in this course will turn around C# and .NET, so here are the versions that were in effect at the time of this recording. Both C# and .NET are notoriously backward compatible. If you're following this course using newer versions, than me from the past sends greeting to you with a lucky guess that everything should compile and work fine on your machine. We will be using BenchmarkDotNet library in several modules. Future versions might break compatibility. Please refer to BenchmarkDotNet homepage for compatibility instructions if needed. I will now outline prerequisites to watching this course. For one thing, this course is not teaching collections fundamentals nor those of generics. If you are an absolute beginner, like not being able to tell how `List<T>` is storing items and what that `<T>` means, then I would suggest that you watch one of the courses on collections fundamentals before stepping to this course. You will have known that there is an interplay between collections and the LINQ in C#, which one cannot ignore in professional programming. However, starting from position that collections seem closer to the ground than a LINQ, this course will not require LINQ. There will be a couple of calls to LINQ in the demos, but only the most trivial ones, I promise. If you do happen to know LINQ already, you will appreciate code terseness that comes out of its use. If you do not, it will at least amuse you to see an alternative notation before you step to a course dedicated to sequences and the LINQ. But I will make sure to avoid the LINQ where possible and only use the most obvious LINQ operators, which should not confuse even the absolute beginner. I will give you a one-minute crash course on LINQ so that you know the operators that will appear here and there in the demos. LINQ operators are extension methods defined on `IEnumerable<T>` so that they apply to any collection or sequence of any T, like this sequence of strings here. Operator `First` returns the first item from the sequence and throws if sequence contains no elements. `Single` returns the single item in the sequence causing an exception if sequence is empty or contains more than one element. Operator `Take` returns, at most, the indicated a number of items from the beginning and ignores those follow. Operator `Skip` does the opposite, skips indicated number of items and returns those that follow. That is enough of LINQ to follow this course, and I will not ask for more knowledge from you past these trivial operators. On the other hand, this course presumes that you feel comfortable about sequences, which means `IEnumerable<T>` and the `yield return` statement. Virtually, all collections already expose members, and most importantly, constructors that depend on

`IEnumerable<T>`. If you do not possess clear understanding of this piece of code, then please read about that first and then come back to this course. Believe me, it does not take more than half an hour to master `C#` sequences and the lazy variation. Finally, this course is not teaching object-oriented or functional design, but it requires functional knowledge of object-oriented programming and design in the least. As a litmus test, try to explain the problem in this declaration. The problem is that this class violates collection and capsulation, making it easy for the outer caller to scramble the data without telling it to the object that owns them. This course assumes that you have understood what I have just said. There will be an entire module devoted to performance testing of logic backed by collections. I will presume that you know already what asymptotic complexity means, like knowing the difference between $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, to name a few. These are synonymous to constant, logarithmic, linear, and log-linear time or space, depending on whether we are talking about CPU or about memory consumption. If you do not understand this notation, please refer to internet resources before stepping to the module on performance. Please note, that in this course, we will develop classes which you can take and put into production code as is. I will be taking design quality very seriously. This course is not meant to be like playing in the sand. In the past 20 years of my programming career, I have already developed virtually every class shown in this course in a significant business application. With that said, let us begin. You will first see the domain model upon which we will develop custom classes and then the story of collections and generics will begin.

Initializing Environment and Projects

In this demonstration, I will quickly walk you through the source code which we will develop in this course. Code, which you will see, depends on .NET SDK, and I will use Visual Studio Code to develop it. You will find recommended .NET versions on this link. At any moment, this page will show you the .NET versions that are under long-term support or considered current. If you chose the latest one available, that will take you to the page where you can select SDK package for your operating system, just follow installation instructions, and you will be there. You can install a Visual Studio Code from this address, select the installation that suits your machine best, and follow instructions which are also straightforward. And so with the .NET and IDE in place, we can step to code, where I will show you the minimal domain model, which will be the foundation for demonstrations to come. This course is about practical use of collections and generics, not about domain modeling, and so the underlying model will

be as small as possible for the purpose. This is the main method implemented with top-level statements which first appeared in C# 9. These statements will become the body of the generated main method. Any local functions you define here would become static methods on the Program class side by side with the main method. There are no using directives here, that is because I have declared global using directives in the Project file. These are the novelty coming with C# 10, Implicit global using directives are enabled introduced in .NET 6. In this course, we will be modeling a company which sells services. Currently, I'm only printing data about workers. This TestData static property getter is serving a handful of valid objects to us. This is the Worker model from the Models class library project. Another C# 10 novelty, File-scoped namespace declaration. This fluent validation may puzzle you. NonEmpty is an extension method defined on this ArgumentExtensions class. When applied to a string, it will return that string if it is not empty or whitespace. Otherwise, it will throw argument exception. Please download source code if you are interested in implementation details. And then, to the PayRate model, it is modeling the money earned in a time interval. It is a record. Records first appeared in C# 9. I need validation here, so I could not rely on auto-generated constructor and properties. But compiler will still generate the rest of the record, namely value type equality. There are similar models for money and currency. Please refer to the downloadable source code for details. I have added Tests project to the solution. There are no tests yet, but there are two convenient classes that can help with the demonstrations, one is the Workers class, which serves valid Worker instances that you can use to experiment. Moving back to main, where I will hit the Run button to see the workers listed in the Terminal. Here they are. Make friends with these people because those are the guys you will be seeing around in demonstrations that follow. In the next clip, we shall cover general division of collections, which will set path for the rest of this course.

Understanding Natural Division of Collections

In an object-oriented language, everything is an object, even a function is an object. And what about multiple objects? Nothing really. Multiple objects are an object, too, a collection. Collections are helping maintain that universal look on all things. When speaking of processes, software operations either apply to a single object or to multiple objects. Either way, we seek universal look on all things where an operation invariably applies to an object, be it truly a single instance of a class or a collection object, if it must. We will do whatever it takes to erase distinction between a multitude of objects and a

single object. We will do that for simplicity's sake. We love simple, streamlined designs. Collections are there to help us attain that noble goal, that is where the story of collections begins and where intrinsic complexities of working with multiple objects step in. Logical division of collections becomes imminent, and there we see three major sorts of them. The simplest form is a linear collection, array, list, stack, queue, that sort of things. Associative collections are more complex. They are associating objects with keys, so that one retrieves an item back by supplying the key value. Dictionaries and hash sets are representing this group. The third major group are graphs, including trees. You will rarely encounter graphs and trees as reusable collections. Expressions in .NET are forming a tree, though you would only reference its root node as a single object. During this course, I will presume that you know these collections already, it will be the practical application of each of them that we will address. In that respect, you will see that seeking simplicity will be your best friend. In the effort to reach simpler designs, we will favor linear collections over associative collections and graphs. If you can just get away with a list or a stack, then look no further. We will favor associative collections over graphs, and we will favor an encapsulated collection over any other. Through combination of these guidelines, we shall unveil the power of abstraction. In this course, we will use all sorts of collections that we will invariably look for their abstract interface, that is where first principles are dwelling. Look at the `ICollection<T>` interface, there are over 50 classes in .NET implementing this interface, most notably, generic list and `ImmutableList` and `Array`. Importance of the `ICollection<T>` interface becomes obvious when we say that it is the linear collection or `IDictionary` generic interface also implemented in dozens of concrete .NET classes. There is even more diversity there compared to lists, and so we find the generic dictionary here, but also the `SortedDictionary` and `SortedList`, as well as all sorts of immutable and concurrent dictionaries. `IDictionary` interface is defining the associative collection in .NET. We can put a list and dictionaries side by side to compare them. List's operation is simple, put items one after another, and they will stay there for retrieval. Insert or remove items that will, but, keep it mind, that performance might not be consistent across implementations. Adding to the end, though, will give you better chances, so people say, but that will also be implementation dependent. Dictionary differs substantially, it promises to associate objects to values used as keys. The purpose of that is usually speed. Other common purposes might be to remove duplicates or, at least, to ensure that there are none. As you can see, there is gross difference between linear and associative collections. They serve different purposes, and they are employed to satisfy different sets of requirements, both functional and

nonfunctional. I'm avoiding graphs and trees intentionally. Graph theory is highly specialized. While this course will explain general-purpose collections, this course will come collections that are either linear or associated. And now, back to a idea that popped up along the lines. You will favor an encapsulated collection over any other. As this course unfolds, you will bear witness to designs which are successfully hiding collections behind an abstract interface. The best collection type is the one you do not depend on. Let that be the motto with which we will step into this course on collections and on generics, too. Collections are generic so let us perform due diligence on that aspect, too.

Understanding the Role of Generics

Collections are organically bound to generics, but generics are also applicable to designs which are not collections. Generics stem from constraint of a programming language, only statically-typed languages need them and have them. Let us get from the other way around from the problem of implementing an algorithm, that will get us to inventing generics beforehand. There are algorithms that apply to only one data, while other algorithms may be general with the option to specialize them to one target type or the other and still lose nothing of their value. Take sorting as an example. You can sort all kinds of things, so long as they are comparable among themselves. You can sort integers or sort cars or sort apples. At the time you implement the sorting algorithm, you do not need to know the type of objects to sort. There are constraints, all right, sorting can only apply to objects of a comparable type. But beyond that request, any type will suffice, except that we have forgot constraints imposed by the programming language. Most dramatic distinction between languages is their division into statically and dynamically type to ones. In statically-typed compiled languages, Compiler will verify type of an object so that every access to an object will be known to be safe at runtime. C# is a statically-typed language, all types must be known at compile time so that compiler can verify statically any assignments, and so we come to a difficult problem, how would a compiler verify types of objects used in a general algorithm? That is where generics come to the table. In statically-typed languages, we write general procedures in terms of a specific, but still not known type, known as type parameter. You will list any constraints required by a generic type or method, that is where compiler takes over and makes sure that all assignments and calls made against objects of type parameter are safe in terms of object substitution principle. We shall walk through all these elements of generics in subsequent modules, and you will have no second thoughts about them by this course's end. Back to the

beginning, this clip has started off with a statement that collections are organically bound to generics. While any class interface or method may choose to be generic or not depending on its scope and purpose, collections will usually be generic, that is because you can collect things of one type or another by applying the same principles and by implementing the same data structures inside. And again, due to compilers demand to verify all assignments and calls, there must be a placeholder-type type parameter upon which collection will operate. It is only later at the time of instantiation or sometimes derivation that we will specify actual type for that parameter, making it a collection of objects of that type or any subtype. Compiler will have its due diligence done long ago, and it will only remain to verify that actual type parameter is satisfying generic type constraints, if any. With that said, we have laid foundation for the rest of this course. We know of collections and we know of generics, and we also know of their working together. In the demonstration that follows, I will quickly show you a notion an idea towards generic implementation. As we progress, we will hit the point where we cannot progress any further without using a collection. And so, in a blink of an eye, by only following customers requirements, we shall come to a full blown generic method which operates on a generic collection.

Understanding the Need for Collections

I will now show you how it comes that we need a collection. Using a collection is not a request, it is a means to fulfilling a request, and the request here is to print the data in form of a grid, not just one object per line. Please observe that request itself does not mention a collection. And it sounds simple, doesn't it? I believe I can solve it in no time. There are the inputted data the format so that they look like a grid. I will need to calculate width of each individual column first, and only, then, to do the formatting. I will use another feature related to top-level statements, local functions. This one will accept the objects and the intended number of columns and return individual widths of all columns into which all data can fit perfectly. Compiler will translate local functions into static member functions on the Program class. You can put them anywhere in file. I'm conventionally placing them at the bottom, after the main method's content. I will first prepare the array, which will automatically be zeroed out. I will then pay a visit to each item in the input trying to fit it into a corresponding column. We need to maintain the running column index as we iterate because the foreach loop cannot tell us that. Inside of the loop, we will convert each item the ToString representation and then take its length. That is how

much horizontal space the item will require, and that number will come repeat with widths of all other items in the same column. Of course, we must advance the column index, taking care to wrap it around the row end. Above, in the main method, we can establish final widths of all columns for a grid with, say, four columns. We are ready to start iterating for data once more, this time determined to construct the GridRows, that will be another local function returning a sequence of strings representing printable rows. I will repeat indexing columns and passing through the data one more time. Only, this time, I will also maintain a StringBuilder containing the current row, that is where this method-specific behavior begins. We need items string representation once again. My code is swarming with a duplication. Please forgive me for that. But we must align the grid nicely, so I will pad every item to its column width, plus two spaces for a gap. If this was the last cell in the row, then I must have just rewound column index to zero. If so, you'd return this row and then clear it so to start the new one. Of course, the last row might be incomplete, so we must check for that and return it as well. This completes the second phase of grid formatting, and we can step up to the calling site where it only remains to walk through all the rows and print them out. Programming is so easy this way. But there is a culprit there. I will run the application first, and then I will get to destroying it with a hidden request, which I conveniently forgot to mention at the beginning. Here is the output. All the data are there and aligned. I'm happy to see it that way. But again, my solution contains a nasty deficiency, which can easily grow into a full-blown bug. Let me show you. I'm walking through the input sequence twice, once to calculate column widths and once more where I constructed the grid. But what if the original sequence does not support multiple iterations? Very often, realistic sequences do not. Socket streams, cryptographic streams, and others of that sort would fail if you tried to rewind them back. Others, like sequences coming from a database will succeed, but at the cost of repeating a potentially expensive query. Let me simulate a stream which only allows one iteration and fails when used the second time. I have just added a class which wraps around the sequence and allows calls to iterate through it, but only once. If someone called GetEnumerator already, this wrapper would throw an InvalidOperationException. This class is internal. I plan to add a public extension method, which will apply it to any sequence. This operator will wrap a sequence into a new single-pass sequence. Now I want to turn test workers into a single-pass sequence. Right at the end of the array initialization, I will say that this array's content will only allow iteration once. You will have noticed this try-catch block wrapping the whole main method's body. I was driving the demo towards this point at which it will

effectively explode, running the application again knowing that it will produce nothing but an exception. If only I had those incoming data stored in a collection, there would be no failure. This demonstration has taught us one half of a lesson, processing data may impose use of collections. The other half is still there to discover, choosing which collection is best for the task and how to use it. As of this module, the task of drawing a grid remains unfinished. In the next module, we will give it full consideration. But before that, let us briefly summarize this module.

Summary

In this module, we have established foundation for the rest of this course. We have put collections into perspective of the main operations. You have seen how collections are helping maintain universal view on all things being objects, even multiple objects can be an object, a collection. And there you have seen that collections are establishing two major models, a list and an associative list. There was the third category there, graph structures, which do not generalize well and, hence, you will rarely encounter a reusable collection class of that kind. The two major classes of collections will remain in our focus for all demonstrations that follow. We have mentioned generics, which are the second topic of this course. Collections are inherently generic. Demonstrations that follow will build on this fundamental interconnectedness of collections and generics. In this introductory module, you have also seen a small domain model with which we shall start, as well as one futile attempt to implement an operation that begs for use of one or more collections. Let us delay no further. In the next module, we will repeat the same exercise, this time, using linear collections, arrays, and a list to cope with these nontrivial requirements.

Using Linear Collections: Lists and Arrays

Encapsulating Collections

Hello, and welcome back to the course on collections and generics in C#. My name is Zoran Horvat. And in this module, I plan to show you the power of linear collections, lists, and arrays. We will implement two examples in this module, which will outline distinct perspectives of collections, linear, or any others. In the first example, the incoming data will be collected into a list, then the items will exchange their places until they assume final positions. After that, the list will freeze. Even though the wrapping object containing the list is mutable, we shall not mutate the list any further because it has assumed its final state for the rest of the object's lifetime, that will open the possibility to build what I will call views or queries on top of the list. These views will return stable and unchangeable results that are even safe for use if needed. In this first demo, you will learn useful coding patterns that help design such a class. The second demo will be quite different. Every call to the object will stir up the list content. We will face the problem of isolating successive call so that they do not destroy each other's data, that will ask for a set of entirely different coding patterns compared to working with frozen data. So the first problem we will address is that of constructing a grid. All this code so diligently constructed in the previous module will go away. The request is to print the data out in form of a nicely formatted grid. In this demo, I will write every single line of code so that you get accustomed to the coding and designing style which I will diligently apply through the entire course. I hope you will not find that demonstration boring. Later, coding practices will put more focus on essential operations on collections and the lesson coding itself. Let's begin then.

Encapsulating a List

My initial attempt was a failure, and I will have to repeat it all over again. And when you must repeat solving a problem the second time, that is already indicative that you have stumbled upon a nontrivial problem which deserves, at least, a class to wrap the new solution in it. Even though this course is on collections, we cannot ignore common rules of object-oriented design. For one thing, you will want to encapsulate your collections. Start off with a class which wraps input data so that it can transform them later. Collection will come later. This design decision to wrap around the sequence is rising two questions. One is the type of data and we shall dismiss that question by turning the class generic. We will not know concrete type of items. That was easy. Now the hard part, what shall we do with the sequence? We have already learned that there will be at least two passes through data. But `IEnumerable` does not promise to allow repeated iterations. One way to make sure you will not

consume a sequence multiple times is to consume it once and remember it in a collection. I have collected the data into a list, but I'm keeping a reference to an IEnumerable. Let me turn this into a reference to abstract list, IList the generic interface, so that at least that fast count property and indexer become available. This is a canonical form of a class which deals with multiple objects. Please note that this class does not tell how precisely it does that. Does it store the objects in a modified form? Does it aggregate objects and store fundamentally different objects in their place? That it does not tell. However, one may remember the design principle to avoid costly work in constructors. If input sequence consisted of a million objects, then we could not ignore the constant memory pressure imposed by this constructor. How do we justify that? Here is my take. I stand firmly with opinion that constructors should be cheap. Their duty is to construct a valid object not to do its work. It is the duty of that object's methods to do the work. This is the method that will work. Those interested in processing will make a call to this method, not to the structure. Calling a constructor is a technical necessity. Keep in mind that we might have to call constructor many times over in a loop, for instance, to populate a collection of objects. And even constructing an object does not warrant a call to any of its methods, some objects come and go without getting a chance to do useful work because something else popped up diverting execution flow around them. Those are the reasons why we normally avoid costly work in a constructor. On the other hand, if constructors did not consume the input sequence, the sequence might change by later time, including a realistic possibility that someone would dispose of it. It is common in design of classes that operate on sequences to consume the sequence in the constructor. All collections do that in their constructors already, that is pushing us to rephrase that earlier principle about constructor costs. We could advise that constructors should only perform minimal work proportional to the size of its arguments, which normally boils down to making a copy of all input objects and trying to avoid any but most trivial transforms. I think we are fine with this design now. A couple of individual requirements have already been satisfied. We have declared a generic class which operates on generic sequence of items, and we have ensured only one iteration for that input sequence by caching its content in a private list. Lastly, we have encapsulated collection so that outer callers neither have access to it nor do they even know that it exists. The whole bunch of useful design decisions hides behind the mere 10 lines of code.

Justifying Decisions

We are now ready to start using the list we created, but I have not explained why it was the list that I chose. My decision came from analyzing various aspects of the requirement. Let me draw them to light here. Length of the input sequence is unknown. Still, I expected a sequence of reasonable size. It makes a little sense to me to format a grid of enormous size so that holding all data in memory becomes prohibitive. That also supports a decision to use a list, which can grow incrementally without knowing its ultimate size up front. We know that we will need to traverse the data column wise and row wise. List sufficient random access to elements would be of help, so I calculated. List indexer guarantees all of one access time. Lists Count property might also be helpful later, especially when you know that it takes all of one time, too. This is a thinking process you will usually apply when choosing the best collection for a task at hand. Contrast the list to an array, which also has all of one indexer and the length property. Chances are that optimizer might make iteration through an array faster than through the list, so why not use the array? There is a convenient link operator, ToArray, for collecting unknown amount of data into an array, but it requires an intermediate storage of a sort before the final array is allocated to an exact count and populated with the data. ToArray operator incurs one additional memory allocation for the entire array. But there is a culprit. Underlying data structure in the list is also an array, which is often underpopulated. Up to a half of the list's memory will be unused, that allows a list to accept more items dynamically. Collecting data into a list will not include the final reallocation and copying, but at the expense of leaving part of the underlying array unused. Should this list remain long lived, its sheer storage inefficiency might become a deciding factor in favor of an array. Still, with no evidence to support this fear, I have opted to ignore danger, at least until more information is available that could revert my decision. One final remark, did you notice that I have not taken any other collection into account, most notably, a dictionary? That is because I favor simple collections over complex ones. So long as a linear collection suffices, given the requirements, I will look no further. Now that this is clear, we should be safe to continue with the list and see how we can implement the missing feature.

Imposing Views upon a Collection

We have a list at hand and an immediate task to calculate column widths, and customers have just added a new request to calculate the number of columns. Caller would supply total width available to the entire grid, plus the desired gap between columns. We have the snapshot of input data in the

private list, and we can safely serve multiple grids out with different widths and gaps. This is important. After encapsulating a collection, methods will effectively be views applied to that collection. They will shape the data into the form most useful to operations we intend to implement. This method format is nothing but a specific view on the list, which returns printable rows. And there will be more. Planning the grid's layout gave me the idea to switch to storing string representations of items rather than row objects. The grid will be strings anyway. Mind the defense against the nulls in the input sequence. There is no reason to keep actual input objects. We shall measure and concatenate strings at will without paying conversion costs multiple times. When you store data in a collection for reuse, try to represent them in the most useful shape, not in their original form. And now we begin building those views. Format will delegate to a more specialized form rows method, which will receive an exact number of columns obtained from yet another method, GetColumnsCount, that operates on the same external arguments. Formatting will also depend on the gap. Now you can see what I meant when I said that there will be a hierarchy of views. All three methods declared so far are views or queries, which convert underlying collection into a specific result, and there will be more of them. Formatting rows means to calculate final widths of all columns, taking the number of columns as input. Only then, can we truly format each row. And we can forward even that call to another specialized view. Formatting rows means to format each row separately. Counting rows is simple, divide total list size by a number of columns and round up. Yet another view, this time producing a scalar value. Now, formatting rows. We want to get cells belonging to the row so that we can join them separated by gaps. Gap will be a string consisting of whitespaces. I want to continue decomposing the original problem because that technique is letting me develop a nontrivial feature by only focusing on one trivial problem at a time, like extracting cells of a row. This well-separated subproblem lets me focus on details that might otherwise slip my attention leading to bugs, like calculating precise bounds for an iteration, which is an inexhaustible source of defects. Now that we are safe from overrunning the list's bounds, we can yield return the elements right padded to their columns width. Would I remember all these details were there not a method where those details belong? Maybe, just maybe. More challenges and more complexities are awaiting us in remaining methods. Completing them will bring another set of design challenges before us.

Completing the Design

I have one idea which will become more obvious as I progress implementing the second method, `GetColumnsCount`, first. This method will attempt different counts and find the largest one where all the data can fit into the allotted screen width. To complete that task, I will presume a helper method exists which calculates the theoretical upper bound for the number of columns. This method will try to squeeze as many items into the first row as possible, keeping record of that row's total width. Please keep in mind that the number of columns must never be zero. Customers agreed that the grid may breach its dimensions if it would be impossible to construct it otherwise. We could see that the number of columns to one to cover that case and seed current width to the width of the first item then, accounting for the possibility that there are no items in the list. This is establishing the initial position and now we can pass through the rest of the first row, trying to add items until we breach the total width. Mine's keeping the first item from the list because we have already included that one in the result. This item will attribute its own, plus the gap that precedes it. If that breaks out of bounds, give up, otherwise increment state variables and step to the next item, whatever remains will be a viable upper bound to the number of columns or at least one. We shall use this result as the initial position for the `GetCountsCount` method. Here is what I planned with it. I wanted to start off with the upper bound and then lower it down while it is producing a grid wider than the argument. This finally brings us to that bright idea which I had a minute ago. Look at this method, `GetTotalWidth`, which still does not exist. It will have to calculate column dimensions so to take their sum. And look at this method below, `GetColumnWidths`, it will also calculate widths of the columns, of course, only without having to sum them up. These two operations only differ in minor details. I saw that coming, and my idea was to propose a third method, which will do both operations at once. In the previous module, that operation was the source of code duplication, which I will try to avoid in this design. To cut the long story short, here is the method. It is returning a tuple containing both the array of individual column width and total width of columns, including gaps. These are all the gaps in the row. For each item in the row, only that item's column and the whole grid might stretch. There is a Boolean parameter, `preempt`, which controls whether to exit as soon as total width breaches maximum so to avoid the needless calculation. We can apply this method in all scenarios that require horizontal attributes of the grid, such as calculating the exact column widths. This call is ignoring everything that relates to the grid width because it is not interested in that aspect. The other consumer, which looks for maximum count of columns, will pass with related parameters, indicating to the method to quit as soon as it establishes

a negative result. This class is almost 100 lines long, which proves that solving problems that involve collections is anything but easy. Before the demonstration, I will also add an operator that wraps a sequence into an instance of this class. By the way, if you are puzzled why this class is called Operators, the reason is in following naming convention from LINQ. LINQ extension methods apply to sequences, just like these extension methods here do. They are referred to as Operators because they produce results from sequences the same way arithmetic operators do from numeric values. The extension method applies to any sequence, wrapping it into a GridFormatter, and requesting it to format the grid. It is wasteful in the sense that we could use the same Formatter object several times. On the other hand, my guess is that calls would only print the grid once and this operator will make it more convenient. Let me show you. Back in the main method, I'm taking TestData and attaching a call to ToGrid. But that operator will do nothing. Method Format ends up calling FormatRows, which yield return's its result. GetRowCount method will execute eagerly, but no rows will come out until someone begins iterating them. We need someone to pull the data so that they start flowing, foreach loop, for instance. There is this extension method, WriteLines, which I have prepared for that purpose. It consumes strings from a sequence and prints them out. I will add a call to WriteLines. That method will foreach for the grids rows, writing them to the Terminal. Yes, the application is printing a nicely-aligned grid. Please keep in mind that the TestData property is still returning a sequence with operator once applied. Absence of any exceptions is proving that the GridFormatter is consuming input data only once, as requested. This completes the demonstration, which has shown the power of lists. List guarantees $O(1)$ amortized time to append an item, $O(1)$ time to retrieve total count of items, and to access an item at random position. We have used a list to construct an algorithm, both time and space linear. And look at the main method, not only that it does not know how we used that list to solve the problem, but it doesn't even know that the list exists. It is data capsulation at its finest. After this exercise, we will put a raise on trial, analyzing situations where they could be of help.

Inventing Shuffling Algorithm

We have completed the first class which operates on the list, and now is the time to move to a different problem domain. By the end of this demo, we shall develop another class which wraps around the list, this time with an entirely different motivation. In this demonstration, I will implement one tricky algorithm, the one that randomizes a sequence of objects. Randomized algorithms are

important, even in business applications. The so-called what-if analysis will sometimes require randomization to simulate future events when facing uncertainty or to search for optimal values, like prices, bids, etc., in a large, possibly infinite, search space. Since this course is about collections, we will implement and use the algorithm which randomizes content of a collection. Here is the task for you. Given a sequence of objects produce it in the output, but with elements appearing in shuffled order. When I say shuffled, it means that every permutation of input objects is equally probable in the output. I will apply the algorithm on the sequence of workers, so as to read them in random order. If I tried again, the same workers would appear, but in a different random order. Can you implement this algorithm for me? Where would you start from? There is a formal solution to this problem, which is even easy to prove correct if you are strong in probability theory. If you are not, pair with someone who is, that is what we programmers usually do to solve a problem that stems from a theory which we do not know well. Using probability theory, we can easily prove that uniform distribution of permutations is equivalent to uniform distribution of items over the N slots, each item having the same probability, one through N , independent from all other items to appear in any position. The request with permutations is tough to implement in code, but distributing items randomly over an array already begins to sound like a program. And there comes the mind bending idea, which quickly evolves into a computer algorithm. In every iteration, we will select one element, remove it from the sequence, and publish it on the output. This element will not appear in any subsequent draw. If we drew the first element from a uniform distribution, then its probability to come out first is one through N . And here is the probability to fail in the first draw, but to win in the second. The element will appear second with the same probability, one through N . Probability to come out in k draw is equal to the product of k minus one failures in a row, followed by a success, one through N once again. We have just proven the shuffling algorithm. In every draw, select one element out of remaining ones, with each element having the same probability to win, then remove the winner from the set of remaining candidates. That is all. This algorithm is known as Fisher-Yates shuffle, and you can read more about it from the Wikipedia article. It was invented in the late 1930s. Interestingly, the algorithm predates programmable computers. The first Fisher-Yates shuffle was done using pen and paper and it had a real application in agriculture. After entailing the proof, we can start thinking of a programmatic implementation. For one thing, if we are to shuffle an input sequence, then we are facing the same problem we had before, not knowing the number of items there are, and Fisher-Yates algorithm is critically dependent on knowing the

number of items remaining before every draw. A collection comes to mind so that we can pick elements out from it until it remains empty. Then, what collection? Removing items from random positions in a list or an array can be costly due to relocation of remaining elements. Removing an item from a dictionary is quick and easy. But what would be the key? Tough questions. If you used to think probability theory is hard, then try writing C# code now that we have entailed expressions. Before you see the solution to this puzzle in the next clip, try to produce a collection-based solution of your own. Of course, it must run in linear time and space. We shall only accept an optimal solution.

Implementing Data Randomization Algorithm

The idea is set, and we can step into implementing it in code. This class will be the home for the Fisher-Yates shuffle algorithm. The first practical question is how to store the input sequence. We can repeat the same protocol, as in the case of GridFormatter, this time holding items in an array. Populating an array can be cumbersome. Collect items into an intermediate list, which expands dynamically. Allocate the array and then copy the list into it. If you allow me to use LINQ, there is that operator, ToArray, which I have already mentioned earlier. It will do these steps for us. And now we have it, only this time the class will behave differently compared to that GridFormatter before, where the private list was read only. Shuffling class will modify the content of the array by exchanging places of items. This method, returning an IEnumerable<T> would output the items from the private array while modifying it. Simultaneous iterations for results of two calls would step on each other's toes, changing the same underlying array at the same time. Bad design. Let us add the request, which will clarify this situation. Shuffling iterations must be isolated. IEnumerable is not the best type to return because it does not warrant isolation. In .NET, IEnumerable is part of the iterator design pattern implementation. The other part is the IEnumerator interface, which specifically warrants isolation among iterations. Now we're talking. I believe that this entire method returning IEnumerable was a bad idea, and I will delete it. Instead of that, I will make this class implement the IEnumerator<T> interface, in order to promise isolation between simultaneous shuffles over the same input sequence. You can reset an IEnumerator either in the middle of iteration or when it is over, and then iterate all over again without causing any damage. That is, at least, how we imagine IEnumerator interface implementation. There are classes in .NET that would throw one reset, others would ignore the call, but not our class. I will retain that plane array representation of data. And the class will also require a source of numbers

that are, at the least, pseudorandom, if not indeed random. I will initialize a random instance using a trick, which I learned from Stack Overflow. This object will progress through the array. It should know the current position, which will initially point outside the array. If anyone calls this enumerator's current property before the first successful call to move next, indexing the array in the current property will fail. It is all by the book so far. Let us get on to implementing behavior. The Dispose method will have nothing to do. That was easy. I will skip to Reset, which is also trivial, just rewind position back before the array's beginning. And now we are finally in the MoveNext method, where the Fisher-Yates shuffle algorithm is waiting to be born. I will first dismiss the case when we have hit the end of the array, just return false and be done with it. We can now advance the Position and focus on selecting a random item for the output. We shall remember that all items starting at this position, and for the rest of the array, are those that we did not pick yet. We shall select one position randomly and that will be the item we select in this draw. And then comes a marvelous idea, we can swap the winner and current element so that the winner takes the current output position. What used to reside in this location will move forward and continue waiting for its chance to show up. Observe how I'm using a tuple trick to swap two elements in place. It only remains to return true to indicate success. This is the Fisher-Yates algorithm at its finest. It takes a page of text to explain two lines of code. Marvelous. I will now include a Shuffling operator. It will return an IEnumerator from a shuffling sequence. I will also provide a helper operator, which advances IEnumerator to the end without destroying it. Someone holding a shuffling enumerator could iterate it, then reset, and iterate again and again, receiving a different shuffle order in each pass. It is important to note that the shuffling class will only hold one array for all passes, never creating nor populating another collection. We have completed the shuffling algorithm, and I cannot wait to see it in action. I will initialize the enumerator and then run five experiments. In each experiment, I want to pick three workers at random. The ToGrid formatter will help condense output. Once each experiment finishes, I will reset the enumerator so that I can continue using it without paying additional cost of initializing its internal collection. Let me show you the shuffler in action. Here are the five groups of three workers, each time drawn from the list of all workers. They look random to me, honestly. You will notice repetitions here and there, but that is what we expect to happen after repeating the experiment. Our Fisher-Yates shuffle looks to be working fine on input data. On top that, it is running in space proportional to the size of the input and in time proportional to the number of items consumed. This solution meets all goals, partly due to using an array as the underlying

collection. That was the last demonstration in this module. Before closing, let's briefly summarize what we have learned.

Summary

In this module, we have seen how to use lists and arrays to implement complex algorithms on sequences. I am intentionally saying sequences not collections because collection is a consequence there. You have learned that collection is useful to satisfy requirements, mostly nonfunctional ones. We have contrasted the lists to arrays from the point of view of collecting objects into one or the other. List is expanding to accept the new objects at the expense of up to a half of its internal storage being unused at any point. Array is tight with no unused locations, but collecting objects into an array may require one additional memory reallocation and copy for the whole collection to become final. Both lists and arrays offer efficient random access to items. With these peculiarities in mind, we have turned to implementing complex features, one that operates on a list and another operating on an array. In the first demonstration, we have used a list to ensure single iteration through input data. You have learned one useful coding technique based on forming a hierarchy of views to decompose a complex problem. These views, as I called them, are querying the collection in all the ways necessary. This technique is useful in operations that do not modify a collection. In the second demonstration, you have learned how to implement a feature which modifies the underlying collection. We had to isolate successive executions to avoid making simultaneous changes to the underlying collection. You have learned one practical trick there, we have attained isolation by implementing the `IEnumerator` interface, which isolates iterations and requires the user to call its `Reset` method before commencing another iteration. Before the end, let me remind you that the second demonstration, implementing a shuffling algorithm, required substantial knowledge of theory of algorithms and probability theory. Working with collections is demanding, you will commonly need to know the theory to be an effective programmer. Do not underestimate the power of knowledge. There will be more algorithms coming from textbooks and tough ones in modules that follow. In the next module, we will see a couple of them. We will put sorted and partially sorted linear collections into perspective of practical features of a business application.

Building on Ordered and Partially Ordered Lists

Designing an Abstract Collection Interface

Hello, and welcome back to the course on collections and generics in C#. After covering benefits of using linear collections, most notably access speed, we will turn our attention to a set of problems that are adding pressure to those same collections. The problem we are about to tackle will require sorting items in the list. The requirement is to implement data pagination with an added complication to sort the sequence before cutting it into pages. More verbosely, the caller will supply an unordered sequence, page size, and a custom sorting criterion. Given these items, the pagination function will first apply sorting to the sequence and only then divide it into separate pages. Please note that we are not just taking the pages out from a sequence, that would be a trivial task already supported by common BCL classes, like List or via LINQs, Skip and Take operators. The request is to keep the sequence sorted through the process of serving pages. That request is common in practice and that is what adds complexity to this task. How are we going to solve this problem? While you think about backing structures and algorithms, I will start preparing infrastructure. I have already separated classes we developed so far into their own sub-namespaces and so the new feature will have a namespace of its own. But it looks to me that pagination will require more than one implementation class. I see that we will need a class that represents PaginatedData source and a class which represents one page. On top of that, I already see two or three viable algorithms to implement the feature. I think it would be wise if we declared interfaces that are defining this feature from the consumer's point of view. The first interface will represent a paginated collection of data. It will be generic because we do not know what kind of objects we shall process. This is the time when we start making wishes. You can design a feature from the outside in and expose members that will be useful to callers without leaking any implementation ideas. What is the Paginated collection? It might know the exact count of pages, why not? This is a nice and useful property from a consumer's angle. What else do we want? Maybe access to objects implementing a different interface, which I will call IPage. We shall return to this interface in a minute. I will also ask for an indexer, which returns a page at

specified offset. And one more idea crossed my mind, why not make every Paginated collection a sequence of pages so that the caller can iterate through them even without asking for their count. This is one nice and tidy interface, which will empower consumers with access to pages and input data. At the same time, please observe that this interface is not irradiating any information about internals of a class that might implement it. We do not know which collection will reside inside. This IEnumerable interface is assuring us that at the least part of this class's duties will be deferred, leaving the room open to implementers to only execute costly operations upon the first call to a member. Now to the second interface, IPage, which is awaiting us. What would you wish to have in this interface? For one thing, I would like to know the content of this page, that is another property getter with implied promise of deferred execution. You should adopt this diplomatic language, this design which speaks in hints and cues. There's no way to force deferred execution of a member. By declaring return type as a sequence, you're implanting an idea that this member will not serve the data at the time of a call, but at the time when an immediate operator starts consuming the sequence. It would also be nice to tell the Ordinal number of this page. Diplomatic speech again, Ordinal means one based. Offset and index mean zero based. Do not confuse these terms in a language with zero-based indices. I want more, a property getter which returns count of elements in this page, but also a property which returns the declared size of the page, in case that this page is under populated. Now I notice inconsistency in the design of two interfaces. Paginated collection is implementing IEnumerable<IPages>, while a Page is exposing an IEnumerable property getter. I prefer consistent designs, and so I will give up this property getter in favor of implementing IEnumerable<T> to access content. I like it more this way. I do not think I will insist on indexer property because I expect pages to be small. Iterating for a page should be a cheap operation, which we cannot assume for the complete collection of an unknown size. With this, we have stated intentions. Any paging implementation must guarantee the features promised by these two interfaces. Please pay attention to how this design will evolve. In effect, we have just declared an abstract paging collection without telling anything about its concrete implementation. There is no substantial difference between this interface and the IList generic interface. Now we will step to constructing an implementation.

Applying Pagination to a Sorted List

We need to design a collection which suits this interface. I think that I will go with a List, but one with its content sorted according to a custom criterion. It makes a little sense to expect someone to pass you the sorted data because, what would this class do then? We want to implement the pagination feature in one place so that nobody else needs to bother with that. And you'll see, paginating data will not be easy, that is the core reason we're encapsulating that feature in the first place. Here is my plan. I want to collect the data into a list, then sort it, and keep it sorted for all future calls. When a request for a page comes, we will instantiate an object implementing the IPage interface, and let it iterate through the underlying list. The trick with this solution is that the past initialization, the list will remain read only. It will be safe to share a reference among collaborating objects so that they can benefit from list efficient indexing capability. I will name the new class, SortedListPaginator. But its internal data structure will not be the SortedList class or anything of that sort, despite the name possibly suggesting that. It will be a common generic list, which I plan to sort myself. For one thing, SortedList requires unique keys, which makes it non-applicable in general sorting problems. It is also very inefficient when you wish to populate it with random, unordered data. In that case, its runtime complexity will be $O(n)$ squared. Contrary to that, populating a common list takes $O(n)$ time followed by a full sort, which takes $O(n \log n)$ time, with an overall cost of $O(n \log n)$, and it adds no uniqueness constraints. Numbers do not lie. We're facing an incoming sequence of unknown size, and we should opt for a collection which guarantees a log linear time complexity. We shall add page size parameter later, after we solve the sorting problem. The usual protocol will commence. There will be a List under the hood, with the constructor paying the cost proportional to the size of its input sequence. But there will be more work to do than just stock the items. These must be sorted, too. Shall we sort them here? Doing so, will incur additional super-linear cost, which grows at a faster pace than the collection. I don't like that design. On the other hand, this class will implement the IPaginated interface, which requests access to individual pages in the sort order. By the way, what sort order? The abstract interface does not care, any sort order is fine. But concrete implementation must decide before sorting. Constructor is in the perfect place to set up all the data switches and strategies that define behavior of the object. This will dismiss the sorting order question. And now the main question remains, how to ensure that sorting cost is paid upon request, rather than here in the constructor. The design, which seems obvious to me, is to prepare a strategy, a sort of a fine delegate, which will serve as sorted list when used, and returned previously sorted the list on subsequent calls so to ensure that the costly operation is only

executed once. Lucky for us, .NET already has a class which models that behavior, the `Lazy<T>` generic class. And here is the strategy that transparently sorts the data. I have given it a decorative name, the naming scheme I like most in situations like this. The static method will return the list with added guarantee that it is sorted. Up above in the constructor, this static method will be the workhorse of the `Lazy<List>` factory function. And now comes the mind bending idea, why hold the list in the object when we have this `Lazy` object that will return that same list, only sorted? You can see that the `sorted data` property getter is doing everything the `data` property getter does, and more. We have an option to construct the list, unsorted, of course, right before the lambda. Then, we can let the lambda capture this list and use it in subsequent operations. With this done, we do not need the `data` property getter anymore. Please make sure to not instantiate the list inside the lambda. In that case, lambda's closure would contain the `IEnumerable` reference, not the list, which means that `IEnumerable` sequence would only be consumed upon the first access to the `sorted data` property's value member. These are the items that are sorted for sure and that will only be subdued to sorting when this property is invoked the first time. But in `IEnumerable` sequence will be consumed right here in the constructor and not later. This is another canonical design which we did not have in prior demos. If you want to separate consuming the input sequence from the costly preprocessing, then this design will help you do it. The time has come to implement the `IPaginated` generic interface so that we can cut the underlying sorted collection into segments and wrap them into concrete `Page` objects. Next step is to design concrete `Page` objects.

Implementing the List Projection Class

At this point, it is very important to notice one aspect of this design. By the time a page is constructed, the list will already be sorted, never to change again. That gives rise to another effective design, constructing a page as a projection onto this list, a window looking into a part of it rather than by copying list segments to new collections. This is what I mean. Starting from the empty `IPage` implementation, I want to wrap around the list, that same list from the `Paginating` class, and then only expose one section publicly. All `Page` objects will share a wrap friends to the same list, but they will serve different sections out, unless their `Ordinal` properties are equal. However, this idea is carrying a grave danger, sharing an `ICollection` object is not safe, its consumers could take advantage of its mutating members and change it. Other instances will access the list, unaware that it is different from what it

used to be. Most notably, the list might change so radically that the other Page instance could not even exist. Imagine clearing the list, that would dismiss the existence of any Page object, and yet they would be there from the times while preconditions to their assistance we're still satisfied. This is the moment when you reach out for a read-only variant of the list. We will request a `ReadOnlyList`, along with the page's ordinal position and its nominal size. Remaining class members will calculate their results, like this `Count` property. Its value depends on the list `Offset` where this page begins. That is easy to calculate. I will also calculate the exclusive upper `Offset` for this page because that will simplify the rest of the calculation. We must cap the ending `Offset` to the list size. `Count` itself will be the distance between two `Offsets`, bounded by zero on the lower end so to avoid indexing errors in case that page finds itself beyond collections and by mistake. Now that we know the `Offset` and `Count`, implementing the sequence of items belonging to the page will be a walk in the park. This method should return the enumerator, which can also be accomplished using `yield return`. With calculated `Offsets` at hand, this task is very simple. Non-generic get enumerator will delegate to this one, and everyone will be happy. The next task is to complete the paginated collection. It will boil down to instantiating projections to the encapsulated list, and that is what we will be dealing with in the next demonstration.

Completing the Design

We need to complete the `Paginator` class. I have added `pageSize` in the meanwhile, that will make it possible to calculate the total number of pages in the list. There will be more work to do to produce a single page. First, we must ensure that page offset is within bounds. This method ensures that offset falls within specified range, throwing `IndexOutOfRangeException` otherwise. `InRange` is the fluent validation extension method from the argument extensions utility class. On successful validation, I'm increasing offset by one so that it becomes an `Ordinal` page number. Now we can move to the indexer, where we will construct a new `ProjectingPage` instance, pass it to the list a valid `Ordinal` number, as well as the `PageSize`. Lucky for me, the `List` class is implementing `ICollection`, as requested by the `ProjectingPage` classes constructor. By the way, if we were to expose an `ICollection` object publicly, it would probably be wise to wrap the list into a `ReadOnlyCollection<T>` object, which also implements the `ICollection<T>` interface, simultaneously removing all mutating members. I'm passing the list privately to a class which is under my full control. It is simple box that I read from and the

IReadOnlyList interface is sufficient to make them impossible. Please note that I am instantiating a new ProjectingPage instance on every call to the indexer. Creating objects, especially short-lived ones, is cheap in C#, that is at least how a functional programmer in me would explain this design. More formally, garbage collector is swiping over short-lived objects without doing any work. It is medium and long-lived objects that require memory-to-memory copy during a garbage collection. Anyway, we shall return to this problem of creating many objects in a later module, where we will effectively measure the impact, challenge this design, and identify its weaknesses. It remains to implement the Enumerator for pages. With calculated PagesCount and indexer in place, creating pages will be the simplest of all operations. Explicit to get a numerator implementation will only delegate to the generic one. And with that, we will complete the class. The class is complete. And if you think it through, you will conclude that this class is nothing but a collection. It is a collection of pages. The fact that it wraps around another collection means little to its consumers. We have just implemented our own tailor-made collection by specializing and adapting features of an existing collection, a common list. On top of that, our lovely collection is generic, applying uniformly to any element type. I cannot wait to see this collection in action. I will have to have the Pagination operator. It will return an object implementing the generic IPaginated interface. In the main method, my plan is to paginate Workers. However, I will sort them ascending by their pay rate. This RateComparer static property on the Worker class is returning an object implementing IComparer<Worker> interface, which we can use to order Worker objects by pay rate. I will iterate through pages and that will give me the opportunity to fetch the data contained in each page. For instance, I could print the average payrate for workers inside the page. This method, AveragePayRate is an extension defined in this helper class. Let us see the output. I'm starting the application, and here it is. There are four pages each, but the last one having the preset size of five items. We can visually confirm that rates are growing page to page, which confirms that workers are, indeed, sorted ascending by their payrate. Everything seems to be working fine, except that there is one sublet, which is hidden below the surface, and which I plan to bring to light now. It has to do with performance. To the demonstrated the cost of the paging operation, I will apply another magical extension method, which I have silently added to the Models.Tests assembly. The Replicate method will take each instance from the sequence of workers and replicate it to the indicated number of times, simultaneously randomizing their payrate 5% around input Workers payrate. If we had 17 workers in the test set, that would explode to nearly a million

objects. I will also make pages larger. This will increase stress on the loop below, the loop which I will dissect to the tiniest bits so that we can analyze what really happens during pagination. First, I want to start a timer, then take the page enumerator. The foreach loop will now become a while loop. Inside the loop, I will isolate the page and then calculate the average payrate. We shall then print the data, including elapsed time. Of course, I must not forget to restart the timer for the next page. Before I run the demo, try to formulate your expectations. Pause the video and try to tell what will happen. And when you resume the video, you will see this picture. The first page has consumed almost all the execution time, making all other pages execute in no time. Theory of algorithms is using the term amortized for this effect. Execution time is being amortized over eight pages. But what if that is not what we wanted? What if we only wanted to consume the first page? Well, things will change and not in a funny way. Full evaluation time will remain, but there will be no additional pages to amortize it. The first page will sustain nearly 100% of the cost of constructing all other pages, but there will be no other pages to make it up. We have a design issue in our hands. A big one.

Understanding Partitioning and Partial Sorting

Now that we have hit the wall, let us analyze what made us fail. The problem was in the first step in the algorithm where we were populating the list. This operation consists of two steps, one was to pull the entire sequence into the list, the other was to sort the full list. It is not possible to avoid pulling the entire sequence. In the worst case, the last element in the input sequence could belong to the very first page, that proves that we cannot construct any page until we placed the last input in its position. However, I do not think that it is mandatory to sort the entire sequence. Looking at the sorting algorithm, the quick sort algorithm which stands behind Sort method on the list, it is a very efficient recurrent algorithm. If you're not familiar with quicksort, here is the quick overview. Given a block of contiguous objects, the algorithm carefully selects a pivot element. Then, it partitions the block so that objects smaller than the pivot go to its left and all other objects to its right. Pivot will stand in the middle to separate the two partitions, that also means that pivot will assume its final position. Then the same algorithm applies recursively to the left and to the right-hand partition, choosing new pivots, partitioning repeatedly, always on smaller partitions than one step earlier, all until partitions become so small that we can sort them using a simpler algorithm, such as selection sort. When all recurrent partitioning operations bubble up, the block will remain fully sorted. That is how quick sort's magic

works. And then we come back to the question of paginating a block, and there we see a crack in sorting logic. Suppose that this is the segment we are interested in because that is a page with an offset requested by the caller. We need it sorted. But do we need objects to the left and to the right of this segment sorted as well? No, nobody asked for them yet. That is where the story of partitioning begins. You can partition an unsorted block and obtain the final location of the pivot. Now you see that pivot stands to the right of the page we need. Why apply partitioning to the right partition if we know that none of those elements will make it to the page? That would be a wasted effort, so we do not make it. If we partitioned the left-hand side, a new pivot might appear on the other side of the page, letting us ignore the left sub-partition this time. Partitioning unfolds in this fashion, we will only process partitions that partly or fully belong to the page. The process continues until we complete sorting the entire page, possibly with a few elements on either or both sides sorted as well. After completing partial sorting of the original block, we will be confident that we have sorted the page, which was the only thing the caller ever asked for. We can construct the Page object and return it in the blink of an eye. The problem is that .NET BCL has now partially-sorted list. We will have to implement it using our own widths. The quicksort variation that I just explained is known as partial quicksort, for obvious reasons, but we will not implement it. We will not have to implement it, that is. In the next module, we will talk about performance optimizations, where you will learn an interesting take on this problem using features that do exist in .NET. Before that, let us summarize this module.

Summary

In this module, we have been dealing with sorted lists. Having data sorted carries great benefits in practice. But to impose a sorting order upon a collection is a tough problem. Namely, sorting takes time and requires all items to be available at the same time. These preconditions can easily become prohibitive with a growing number of objects. In this module, we have been dealing with the problem of partitioning a collection with an added constraint to make all partitions sorted. We came to a solution, which we have applied to the problem of paginating a sequence. Partitioning and sorting often go hand in hand and they are, as we have seen, tough problems, hence the observation with which we have started this module. When solving problems, you should be prepared to solve them in more than one way. One solution would behave well on a subset of perceived use cases. Another solution would behave well in other use cases by building upon a different underlying design and imposing different

constraints. An idea came to mind to define an interface and abstract type, which sets promises given by any concrete solution we might produce later. Concrete classes would encapsulate different ideas and implement the interface to guarantee all functional requirements promised by the interface. Nonfunctional requirements are where the problems are lurking, and the core reason we are envisioning multiple solutions in the first place. In the pagination demo, we have successfully designed a class which is performed under presumption of so-called amortized complexity. We are expecting the caller to walk through all the data so to amortize the exceptionally costly first step through data. We are in trouble when this assumption does not hold, as you have witnessed that tragic incapacitation of my partitioning collection when a caller only used it to extract one page. You have learned the basics of partially sorted lists, which apply so-called partial quicksort to only sort specific sections of the list. In the next module, you will learn the basics of performance measuring and optimization, and that will be the opportunity to return to the question of partially sorted lists.

Measuring and Optimizing Performance

Measuring Performance

Hello, and welcome back to the course on collections and generics in C#. In this module, I plan to show you methods of measuring performance of code that operates on collections. We have been struggling with sorting in the previous module, but we have not measured its performance. Measuring how long it took to sort a list would be of little help when there is no alternative solution it to compare to. In this module, we will use an alternative solution to measure using the measurement itself to guide the making of that alternative. The benchmarking tool of choice of many programmers is `benchmark.net`. That is a lightweight library, which I will warmly recommend. I will add the `BenchmarkDotNet` package to the Console project, and then we will begin. Here it is. I wish to create a couple of Benchmark methods, but I am not a fan of polluting the `Program` class with them. You can create a separate class for Benchmarks, assigning it a descriptive name identifying its purpose, a `SortingBenchmark` here. There is little to do to create `Bbenchmark`. You will usually isolate a piece of

logic which is critical to the operation that is causing concern. It was the sorting operation that dominated the execution time, so I will add a Benchmark with only measures sorting speed. Inside a public class, provide a public method which does the work of interest, only be so kind as to apply the Benchmark attribute defined in the BenchmarkDotNet.Attributes namespace. And there we get to the point, the operations we want it to measure. We will obtain a sequence of workers, like 1 million of them, and collected them into a List. I'm returning the list to avoid any optimizations that might remove the local variable affecting the measurement. Although, honestly, I cannot tell any rational reason for the optimizer to remove any of the objects in this operation. It is my habit to force use of all the variables in measured code, as I have seen optimizers surprisingly removing parts of my code, leading to a false measurement. Benchmarking can be a tricky business. The sorting operation is the crux of this Benchmark. We use the Worker.RateComparer for the sort order. This completes one Benchmark. It remains to pass it to the runner. Use the BenchmarkRunner class from the BenchmarkDotNet.Running namespace, call its static run method, and tell it which class contains public methods with the Benchmark attribute. The BenchmarkDotNet framework also requires release built to a run. But this is not all I wanted to do. I wanted to compare two distinct solutions to the list-sorting problem. Even though I promised not to use LINQ in this course, I will still do it here in this situation because I need the second implementation to compare with the official one. I will use the OrderBy operator from LINQ. It accepts a lambda, extracting the value by which to order the input sequence. And I will collect the results into a list using the LINQ's ToList operator. The Sort method is accepting an IComparer generic interface, while the OrderBy operator receives an expression that yields a comparable object. It looks like these two methods differ in the way they compare items during sorting. There are a couple of details to add to make Benchmarks more relevant. First, I want to collect results in multiple points. You can do that by declaring a read/write property adorned with the params attribute. Next, it makes sense to add one Benchmark, which is only performing infrastructure operations, those that you cannot affect that which contribute to total execution time. Collecting objects into a list is inevitable in both solutions. I would like to know a percentage of total time consumed by collecting into a list because I cannot eliminate it, and I do not want it to surprise me. For instance, if it turns out that an inevitable operation takes 90% of time, then optimization could only affect the remaining 10%. I am sure you would lose any interest in optimizing if that is the case. By the same token, do not be afraid of making such a discovery. There is one simple, but powerful truth, in

research, no is a valid answer, too. If you can demonstrate that one design will not satisfy performance requirements, no matter how hard you try, then you have done a tremendously useful work already. You will direct the remaining energy into inventing a different algorithm, a different data structure, anything that can turn the tide into your favor. We are almost ready to run the Benchmark. The last touch will be to mark one of the Benchmark methods as Baseline against which the others would compare. In my experiment, this Benchmark is a natural Baseline. I will run the application from the terminal window because Benchmark is exceptionally verbose. On the other hand, I find beauty in what it prints out while burning the CPU. This run will take a couple of minutes, and when it ends, the results will be priceless. The table displays a lot of data. Rows are grouped by parameter value. The most important columns are the parameter value, mean absolute time expressed in microseconds, and times relative to the baseline. Execution times are highly dependent on actual hardware, that is why I have used the Baseline approach. Runner has included an additional column, indicating relative costs of Benchmarks. That measure is likely to remain stable, irrespective of the underlying hardware. The result is that the LINQ operator is consistently slower than the list by a small margin. It is because it turned out that the LINQ implementation reduces to the same code as we had with the list. This experiment has failed, but not for long. I have another idea in mind.

Understanding Optimization via Redesign

When you compare measuring performance of an implementation, that will often only mark the beginning of a longer journey. It is so, because you will rarely ever measure performance without being under pressure to improve it. Measurement results will then point you in the direction of viable improvements. I say viable because success in this journey is anything but guaranteed. Here is what I mean. The experiment with the LINQ, which did not turn up as a savior, is still changing perspective because we apply LINQ operators differently compared to imperative code. This, what I am about to tell, will not help us solve the problem of sorting performance directory, but it will be an important lesson when trying to improve performance. That lesson is about outside of the box. LINQ operators are notorious for their flexible behavior. If you followed `OrderBy` with the first operator, that would mean that you request the smallest element in the sort order. This is one possible answer to the frequent question in job interviews, the wrong answer, to be precise, unless you know a secret. In recent releases of LINQ, `OrderBy` followed by a parameter as first operator, will not order the

sequence. That is the result of a clever optimization. This could be an ace up your sleeve in the next job interview, so listen carefully. Suppose that we used `OrderBy` to sort the sequence ascending by a comparable key. That operator returns an `IEnumerable`, enumerating items in the ascending order. We can assume that it will use a list for that purpose, taking $O(n \log n)$ time and $O(n)$ space, and all of that for the purpose of picking the first element from the result. That is wasteful. When the first operator notices that its operand is in order `IEnumerable`, it uses an internal mechanism of LINQ to ask that `Enumerable` for the first element in sort order without sorting itself. That is the same result as one would obtain after full sort. Only that this alternative operation will take $O(n)$ time and $O(n)$ space. Wonderful. But there is a culprit here. The first operator has an overload, which receives a predicate so that it returns the first item that satisfies the predicate. That same optimization is off limits for this overload for a very peculiar reason. Without the optimization, `OrderBy` would apply a full sorting algorithm, which would yield a sequence sorted by the key. The upcoming first operator would apply the predicate to one element at a time and return the first sorted element for which predicate returns true. And now we start noticing differences when executing with and without the optimization. The non-optimized variant would skip items from the sort order, while predicate returns false, and then take the first element unconditionally. But now, think of this same operation with the optimization included. It would pass through the entire sequence, applying the lambda to each element, and selecting, among those for which lambda returned true, the smallest item according to the sorting criterion. Is this a last transform equivalent to those that preceded? Think carefully. For one thing, their results will be the same. So we could hastily say, yes, these expressions are equivalent. But there is a subtle difference, the optimized form would execute the lambda as many times as there are elements in the sequence. But the non-optimized variant will stop at the first item that satisfies the predicate. Just for a moment, one could think, oh, who cares? But then we remember at least two use cases where someone would care. For one thing, the predicate could be an expensive one. What if multiple invocations on the predicate will cost way more than log linear sort plus a few predicate invocations, that would cause a nonfunctional defect because our performance optimization would turn out into a performance penalty. And worse is yet to come. What if the predicate had side effects? In the non-optimized variant expression, side effects would show up fewer times than in the optimized one. The two variants do not produce the same effects, even though their return value will be the same. That is why designers of LINQ have only implemented that optimization in parameters first and not in the overload. This

example is teaching us valuable lessons. While optimizing code, we may introduce nonfunctional changes that may have adverse effects on existing users. And worse yet, an optimization might not improve performance universally, but only under certain conditions. If that is so, then we must advise the callers through documentation, how to select an implementation that is right for them. While we can transparently substitute the optimized version for the prior one, we should only do that on our own if that will not introduce breaking changes. Otherwise, it must be the caller's decision which implementation to call. Users who know that their situation falls within the documented subset will abandon the original solution in favor of the optimized one. Now that we know the rules, let us get back to solving that list-partitioning problem.

Using Benchmark to Explore Viable Redesigns

As this course progresses, I'm constantly battling two extremes, whether to teach you as much of the theory of algorithms as possible because that is what makes working with collections fast or to read the course's title as it is, collections and generics, and to stay within premises of what .NET BCL is giving us, plain collections. The problem with the first approach is that you might find it too difficult to follow through. The problem with the second approach is even worse, using BCL collections without applying proper domain modeling is the road to disaster. You will design and then use wrappers around collections. Your custom classes will implement business-related operations while only relying on common collections as a data store. The topic of this module is performance optimization pertaining to the last problem we were solving, pagination or list partitioning. You will notice that we are constantly using a plain list for its fast access and great sorting algorithm, but we are still struggling with sorting performance. My current plan is to explore one narrow use case and try to construct a different, more performance solution in that use case. Here is the new benchmark I have prepared. It will measure the cost of taking the first page ignoring all others. That is the narrow use case which business analysts have discovered. There is that old joke saying that the best place to hide content is at the second page of search results. In our imaginary application, more than 90% of all requests give up iterating after the first or the second page. Business wants us to take that information and build a faster service. We plan to experiment with data structures that will isolate the first page in sort order and ignore the rest of the data. The Baseline method is only populating the list, that is the fundamental boundary below which I do not expect to be able to go, no matter how clever

my code might be. The other Benchmark is extracting the first page from a fully sorted list. That model is the bad idea, the wasteful idea, the one I wish to beat by a large margin. There are two parameters, number of items and size of the first page requested. Parameters can have priority in BenchmarkDotNet, lower priority coming first. The output will appear sorted by the SequenceLength first and then by the PageSize. This is the solution inherited from prior experiments. It is sorting the list and then removing all items past the first first page. It is simple, straightforward, and inefficient in that it sorts more data than needed. We want to beat its running time by applying the following idea, add incoming objects into a sorting collection, trimmed to the page size, throwing all excess items away, knowing that they will never make it back to the first page. Code is speaking more than 1000 words, I will be using the SortedList class which requires the sorting key. Workers will be their own keys, compared using the pay RateComparer. The problem it sorted list is that keys must be unique. Just this time, I will make an entirely unrealistic assumption that workers have unique pay rates. This helped property will guarantee that for us. I will be able to populate the sorted list with workers observing its hard constraint. Please note that I am doing all this just to be able to measure the effects of this idea not to construct a production-ready code at this stage. And then comes the novelty, Workers added so far will remain sorted ascending by day pay rate. Every time the collection size breaches the page size, remove the last item, which is the largest one in the sort order. That worker would never make it to the smallest page after discovering enough workers with the lower pay already. Repackaging items into the list will be a bit engaging. Sorted the list is not the most programmer friendly type. Despite Add methods, quadratic time on the sorted list, the list will be very small, irrespective of the sequence length. I hope that will allow it to compete on equal footing with the log linear sort method of the common list when the list is a large. I need to take workers with unique rates in the Baseline test, too, to make it a fair measurement. It will take a couple of minutes to run the Benchmark, so I will just show you the results. The BenchmarkDotNet creates a subdirectory where you can find results. Here they are. I will open the HTML version. You can see that partial sorting is significantly faster than full sorting. This experiment chief to establish is showing potential to reduce execution time. Keep in mind that sorted list is far from a desirable choice here. It runs in quadratic time and adds the uniqueness constraint to keys, making it non-applicable in general case. But even with a clear performance penalty, it has outperformed full sorting by a large margin. Before this course ends, you will learn how to design a collection, which is more suitable than the sort of list. It will run in

log linear time and add no uniqueness constraints. Designing that collection will require skill and effort. Success in this benchmark is a prerequisite to undergoing that development. But again, that effort will incur additional development costs. Can we postpone it at least if not avoid all together? In the next demonstration, we will investigate where the existing operations are spending extra time, hoping to improve running time of an existing design without having to develop an entire new set of classes.

Picking Low Hanging Fruit

I have indeed inspected the existing code. Here is what I have found. The sort method is sorting ascending by pay rate return from Worker's rate property. That property is returning the PayRate object. This object, as you can witness, is implementing the `Comparable<PayRate>` interface, so we can use it as a sorting key. That is fine. What bothers me is the implementation of the `CompareTo` method. Since this object is modeling money divided by time, the `CompareTo` method is normalizing durations first and then comparing money amounts. Duration normalization causes construction of entirely new Money objects for the task. Bottom line is that every call to `CompareTo` onto PayRate objects will cost us, now listen, two double-precision divisions, two decimal divisions, construction of two new objects, and only then a single decimal comparison, hiding behind this inner call to the `ComparedTo` method on Money objects. That is a disaster. On top of that, all these objects are reference types, which incurs many small allocations that are subject to garbage collection. It looks like my domain model is not susceptible to sorting, which is a pity, because sorting is a fundamental operation in programming. You can expect sorting to rear its head behind every corner in customer's requirements, believe me. Then how do we resolve this issue? One way is for precalculating values when it is likely that the caller will request them repeatedly. We are possessing evidence that the sorting procedure is repeatedly requesting hourly rates for the same Worker object. Why not calculate it once and store it in the object? We will calculate this property's value, the final value, precisely once in the constructor, and store for future use. Please observe that we are trading CPU for memory with this change. Memory footprint of the whole collection will increase slightly, saving a bit of CPU in comparison operations. More precisely, we are saving CPU for caching repeatedly requested objects. There is one additional performance optimization available in C# 10. Sometimes your records will be small, like Currency in this model. Starting from C# 10, you can also define struct records. And to make them immutable, declare them as read-only, along with all instance level fields. This is new

syntax not available in C# 9. I hope to be able to demonstrate speed up by adding a new method to SortingBenchmark's. I will delete this Benchmark based on LINQ and prepare a brand new one, which will sort workers by hourly rate, which is Money, not by the more complex PayRate objects. This Benchmark will populate the list and return it in the end, just like the first one. But the sorting step will be different. This time, I will use a Comparer built from lambda, which compares HourlyRate's rather than PayRate objects. I also want this new implementation to be the Baseline so that prior implementation compares to it, not the other way around. That will supersede the old Baseline declaration. When I run the Benchmark, the data will display the new order. Here are the results. By trading CPU for memory, we have succeeded in driving execution time down by a factor of three. This is wonderful news. They're even removing the imminent threat of having to implement the partial quicksort algorithm. Sometime in the future, we still might have to implement that complex algorithm. But thanks to this quick patch, that time will not be today. Let us put new code into motion and that will help us decide. Here is the last production code we had at the end of the previous module. Back then, we replicated workers 50,000 times each, making 850,000 of them total. We used the while loop to measure performance of each page in isolation. Please observe that this code is still using the old Comparer. The only difference this time is that Currency is a struct, not a class. That will affect the outcome positively, but not decisively. I have started the program and here it is. Please remember this result here, that is the cost of generating the first item on the output. All other items are coming out without added costs. Let's fix the Comparer then. In the Worker class, the RateComparer will effectively compare hourly rates, which are Money objects. If we run the application again, we will witness real improvement. Here, the time is down by a factor larger than three, which is even more than what the Benchmark had predicted. This experiment is important in two respects. One, we succeeded in reducing the first step penalty to a level, which has cooled down our customer's insatiable appetite for speed. We are safe from having to implement the dreaded partial quicksort algorithm for now. The second important outcome is that performance improvement we observed in a production grade application is consistent with improvement reported by BenchmarkDotNet. This proves that benchmarking is a reliable tool, which allows us to run experiments in isolation, and to make qualified design decisions supported by measurements. With this successful run, we shall close this module. Let's briefly summarize what we have learned.

Summary

This was an intensive module in which we employed the BenchmarkDotNet library to measure performance of algorithms applied to a collection. Measuring performance is important because operations and collections usually cost more than other operations in code. Sheer volume of data that may pass for a collection will likely make it a prime suspect in any bottleneck or performance investigation. You have learned how to create and run Benchmarks on isolated pieces of logic that operate on a collection. You have seen how to construct Benchmark methods and how to measure their execution time. You have seen how to set up their varying parameters so to construct measurement tables that will guide your future design decisions. Beyond measuring current implementation, you have learned to use Benchmarks to validate isolated ideas, pieces of algorithms, so to conclude whether there is enough room for improvement or not. Last but not least, important, you have learned how to measure the impact of small local changes in current implementation to try to improve its running speed without undergoing major redesign. Any local changes that can help make code faster are welcome when contrasted to expensive and often uncertain redesigns. As the very last demonstration in this module, you have seen an implementation which incorporates changes that pass the trial using a Benchmark. You have witnessed runtime improvement of a production code, which is on a par with what Benchmark has predicted, that is acknowledging benchmarking as a viable tool in optimizing performance. With this, we're ending the first half of the course in which we have covered the mostly theoretical aspects of working with collections. The goal was to empower you with the tools required to design algorithms and collections. A large chunk of fundamental knowledge is still missing, that of generics. Before we step into the last part of the course, where we will design real-world collections that will shine in your business applications, we need to cover the topic of generics. That is what we'll start doing in the next module.

Using Generic Types in the Object Model

Understanding Variance

Hello, and welcome back to the course on collections and generics in C#. In previous modules, you have seen linear collections in action. And more is there to show, but we had to stop for a moment to discuss inner workings of generics in C# before we can continue. Let's begin with one tricky topic, generic variants. Generic types are all about substitution. If there were no need to substitute one type for another, there would be no generics. And we mean substitute as in the object substitution principle. A reference can only receive an object of a subtype of the reference's declared type. Subtype is either that type or a type which directly or indirectly derives from it. Compiler must verify that the object assigned to the reference will have all the members the reference type has and so it imposes a constraint on an assignment. It is all about the compiler's ability to assert the subtype relationship and then generics come, and the bar is raised higher. Testing subtype relationship between generic references must also take generic type parameters into account. Out of the blue, subtyping has become a multidimensional problem. What is that variance thing then? Its name comes from physics, where we observe how changes in one axis of the problem correlate with changes in the other axis. If they change in the same direction, the problem is covariant. If they change in opposite directions, the problem is contravariant. In programming, the axis can display types in a direction of less derived to more derived ones. Along the horizontal axis, we will mark generic type parameters, like object and string, with string standing to the right because it is more derived. Vertical axis will display generic types, for example, IEnumerable. But which is the subtype of which? To answer the question, we will observe what that instance is doing. IEnumerable of string will return string objects, which are assignable to object references. Therefore, if we assign an IEnumerable of string object to an IEnumerable of object reference, everybody will be happy. IEnumerable of object behaves as base to IEnumerable of string, telling us that IEnumerable is covariant on its generic type parameter. Let us consider another generic interface, IComparable, then. Its method will receive an object. If we assign an IComparable of string object to an IComparable of object reference, hoping for the same success as before, then the caller would pass an object instance to a method which expects a string. That would violate the object substitution principle, so we conclude that it must be the other way around. We could pass a string to a concrete method which expects an object. And so, we conclude that IComparable of string is less derived than IComparable of object, even though their type parameters derive in the opposite direction. IComparable is contravariant. For practical reasons, C# only allows variants in interfaces and delegate types. It is not likely that that will change any time soon. Since

covariance is an attribute of return values and contravariant is that of input values, C# is using the keywords out and in, respectively, to indicate them. A generic type parameter, which has neither of these is said to be invariant. Variance is confusing programmers. Even the authors of C# were confused in the early days of .NET Framework 1.0 when they introduced an infamous feature known as array covariance. You can assign an array of strings to a reference to an object array. C# considers array type covariant. But look what it will let us do, an insane operation. A DateTime is an object, right? So the compiler will not complain. That is the problem because this assignment will cause an exception at runtime. The team did not repeat the same mistake in .NET Framework 2.0, where generics, including generic collections, were introduced. Generic collections are invariant, which means neither covariant nor contravariant. This line will not compile. In a practical sense, covariance is aligned with how our eyes are seeing code. You have seen that IEnumerable is covariant and, indeed, building on its covariant nature is easy. I find nothing confusing in this piece of code. Contravariance is a different story, though. Let me show you what may happen if you try to apply one typical contravariant type, the IComparer generic interface.

Demonstrating Variance

I have made a change in the domain model. This is the new class, Employee, and it is more general than the Worker. Employee holds a non-empty name and exposes a suitable comparer, which only compares the names. Worker class is now derived from Employee. It also exposes its own comparer, which delegates to baseComparer first and only breaks ties by comparing its own properties as a second order comparison criterion. And then comes the substance. I will isolate five workers first. Then, I will duplicate each worker, setting a different payrate to make their comparison more engaged. I will also shuffle the workers. This is the sequence of Worker objects. But I want it to operate on Employees. This assignment is legal because IEnumerable is covariant. You can witness that the compiler has raised no issues. Printing the employees out would go smoothly. Here is the output, and you can see that every name is appearing twice. Let's raise the bar. The second sequence can also contain plain employees mixed with derived Worker objects. I will take whichever five workers came out first and construct employees with the same names using a bit of complex LINQ expression for the purpose. If you are new to LINQ, the result will be a sequence consisting of 10 Worker objects and five Employee objects. I will now shuffle these 15 objects and collect them into a list that we'll use later.

When I run the application, you will notice names without payrates. Those are the base class instances, which do not contain such a field. This is the moment when we remember that both classes have their comparers, which we can use to sort the sequence before print out. Let me collect items into the list first and then we will sort them. I will use the `EmployeeComparer` because we're operating on the list of `Employees`. When we run the application, we will witness, that while people appear ordered ascending by their name, the sort order is not taking their earnings into account. That is normal because we use the comparer which does not even know that such a thing as `payrate` exists. Then, look. What if I tried to fix the sort order by substituting the `Worker's` comparer? Well, the compiler has something to say about this situation. The `IComparer` is contravariant. `Comparer of Workers` is not a subtype of `comparer of Employees`. It is the opposite. `IComparer of Workers` is a super type of that of the `Workers` own super type. Confusing, huh? But if you think this through, you will see that it makes sense. This comparer will receive any object that it encounters in a sequence of `Employees`, not only `Worker` instances alone. `Employee` instances will also appear in who knows what else in the future, that is the trait of contravariant types, and it comes hand in hand with another trait. It requires careful thinking. Contravariance is a dangerous tool. Let me show you. This comparer will work fine when applied to a sequence of `Worker` objects. Compile time error is gone, and we can run the application. It has sorted workers by name, and only then, by `payrate`. Even the last pair, which used to show up inverted is appearing in the correct order this time. But now comes a surprise. We said that the comparer is contravariant, didn't we? Then we can sort `Workers` using the `EmployeeComparer` as well. The `Worker` class is inheriting static `EmployeeComparer`, so everything is correct both syntactically and semantically with this call. `Workers` look sorted correctly, first by the name and then by the `payrate`. But that is just a coincidence, revealed by the last pair, which is inverted. Was that our intention? Did we really want to ignore `payrates` as the secondary comparison criterion, or is this a plain bug? A bug which almost slipped away because it was only one out of five pairs of objects that revealed it. Working with contravariant objects is a dangerous business. It gives you a certain power, but sometimes accompanied by unintended behavior, a.k.a., bugs. I cannot remember the last time I declared a contravariant generic type. There is one reason for the limited usefulness of contravariant types. Contravariants affects input parameters, objects that we pass to methods. Well, guess what? You can always pass an object of a derived type. The object substitution principle is already contravariant on method arguments. It is often a lose, lose feature, at least in my

opinion. Anyway, it is there. And if you ever encounter a genuine situation which requires it, feel free to apply it. My point is that any such occurrence will likely be a false alarm. Covariance is a different story, it is widely applicable and helps produce more expressive types. The next demonstration will be the opportunity to start designing one useful covariant type.

Designing a Covariant Generic Type

In this demonstration, we shall define one covariant generic type and this concrete implementation. That will be a collection, so let me create a namespace for collections first. We will build on prior examples with a list of sorted objects. I'm careful not to say `SortedList` here to avoid confusing it with the .NET BCL generic class called `SortedList`, which we have met in the previous module. What shall the `IOrderedList` do for us? For one thing, I want it to inherit from my `IEnumerable<T>`. It will be a sequence of items, anyway. Next, I want quick indexing. I'm including an indexer that maps index into an object of type `T`. And the last feature I wish for is to efficiently read collection size. This completes an abstract definition of an ordered read-only list. It applies to any type `T` reference or value type all the same. But this interface here is special in one respect. Look at its base, `IEnumerable<T>`, I will open its metadata. Here is the important detail, generic `IEnumerable` is covariant. That is what this keyword, `out`, is telling. It is covariant because all it is doing is to return objects. I want that same feature in my beautiful interface. All it does is to return items of type `T`. There are no obstacles to declaring it covariant. Some IDE extensions will even warn you of a missed opportunity to specify variances. I will be frank with you. I have a habit of declaring covariance, but admitting to declare contravariance. You may choose to ignore this quirk of mine and go on with both covariance and contravariance declarations by the book. And now, we can take one step back to notice that this interface is in every respect identical to an already existing interface, `IReadOnlyList<T>`, which in its turn inherits from `IEnumerable<T>`, defines these two members, and which is covariant. Should we remove our interface then or just inherit `IReadOnlyList<T>` instead and remove all the content? What would be the purpose of an empty interface? I can tell you two viable reasons right away. One is the so-called marker interface. A class might implement an the interface without having to do anything about it, which means that nothing is changing syntactically. But that would incur a semantical change because now the class says that it is designed for a context where that interface has a semantical meaning. For instance, you can imagine a method which is not expecting just any `ReadOnlyList`, but the one which

promises to contain ordered data. `List<T>` is implementing `ReadOnlyList<T>`, but that would not suffice to call such a method. You will learn constrained generics by the end of this discussion on generics, that is also a good place to use marker interfaces so to constrain generic type parameters to only the types that implement this interface, however empty it may be. Either way, you can use an interface defining no members to pass semantic information throughout the design where there is no syntactic means to accomplish that goal. The other more, down-to-earth reason is to create a placeholder for future members. For instance, I might want to extract a range of an ordered list. That makes perfect sense because this is the ordered list, and so the range will remain ordered. Someone will appreciate that. You will remember that a similar public method already exists on the generic list. This becomes especially useful when combined with default interface methods feature from C# 8, where an interface can expose a fully operational default implementation. Existing implementors do not have to change unless they have access to a more performant implementation of their own. Default implementation will effectively add a method to all concrete classes without the necessity to change them under the risk that this default implementation might not be the most optimal in all cases. Namely, some concrete classes might know better and it is advised to still implement the new method in those classes. But I have an even better idea, if range is ordered already, then why not return it as an ordered list? That would require instantiation of a concrete class which does not exist yet, and so default implementation is off limits. But my prior point remains, an empty interface has defined a placeholder for a future extension. I think that this method will be an exceptionally useful one in demonstrations that are yet to come. By the way, I mentioned that this interface applies to reference and to value types uniformly, that will be a riddle for the compiler to solve. Do you know how generic types are compiled in .NET? I will tell you that next.

Understanding Compiler's View on Generics in .NET

Compiler is analyzing all the fields and methods on a class, including inherited ones. Methods are accessing fields, and so they need to know relative locations of fields inside the objects layout. That is why the compiler needs to know the size of each individual field. In nongeneric classes, that is straightforward, it becomes more engaged in generic types where some fields are of a yet unknown type. Their space and object layout will depend on their actual type, which is not known in advance. If type parameter were a reference type, then it would become easy again because all reference types

have the same size. How about value types, then? They can be of any size, and the compiler must postpone the process until the size of the type parameter becomes known. That will happen at the place where we instantiate the generic type. For instance, a `List<T>` remains in a half-baked state until we declare an instance, such as `List<string>`. `String` is a reference type, which in a 64-bit build, takes 8 bytes. We will be handed over a generic `List<T>` with exactly 8 bytes allocated in every place that mentions type `T`. Underlying collection in the list is an array. Should this dimension be 100 items, then 800 bytes of memory would be allocated. And then we allocated the list in another place, this time as a list of `DateTime`. `DateTime` is a struct and its size is also 8 bytes, how convenient. Compiler will ask for a list with 8 byte slots again. Well, surprise, it will reuse the list description prepared earlier when a `List<string>` asked for it, only this time storing `DateTime` instances into those same field slots. It will amuse you to learn that the `Lists<string>` in the `DayTime` will only require one type template in memory. Should you declare a list of in 32 later in code, then that would ask for one additional version of a compiled list, the one which reserves 4 bytes per instance of its type parameter. Not perfect, but still a very efficient method of packaging types in .NET. Nevertheless, this should not bother you. Feel free to instantiate generic types in any way you find useful. Compiling them into binary code is the compiler's job to do, anyway, not yours. This completes the introduction to generic types in C#. That is a prerequisite to using more advanced features of generics, which we shall discuss in the next module. Before that, let's briefly summarize what we have learned so far.

Summary

In this module, we have started from a concept which is a confusing one, variance. You have seen that variance defines subtype relationship between generic types. When a subtype of a type parameter generates a subtype of the generic type, we're talking about covariant generic type. When it generates a super type of the generic type, the type is contravariant. Common objects are bound by the object substitution principle which determines which object we can assign to which reference. The same object substitution principle holds when we instantiate generic types, so we need to know subtype relationships among generic types. That is what variance is resolving. You have learned how to specify variance on an interface and how to take advantage of it when using concrete generic class is implementing a variant interface. That was just enough for us to define a nice covariant generic interface still with no class to implement it. From this point onward, we will step on to concrete generic

classes and generic methods. I hope that you have enjoyed the ride so far. A lot of code is awaiting us in the next module and a few more advanced features of generics, too.

Designing Generic Types

Implementing a Concrete Generic Type

Welcome back to the course and to the module in which we will implement the generic interface designed in the previous module. I will create a class with an indicative name, `FullSortedList`. I will not bother to tell that this list is read only, its members will reveal that. But I will bother to say that this class represents a fully sorted list, as opposed to partially sorted one. That, between the lines, is telling that obtaining an instance will cost you $O(n \log n)$ time and $O(n)$ space. I'm careful to communicate this information clearly. You can define a generic class even without an interface. There are at least two practical reasons to implement an interface. One is when the interface defines only a subset of class members as in the interface segregation principle. The second viable reason is when there are other alternatives to implementing that same interface, so to let consumers depend on either of them via dependency injection. I think that there might be other implementations of the order list later each based on a different premise, and that was the reason I have started this development from the interface in the first place. Please observe that we cannot specify variance in concrete classes, it is only applicable to interfaces, and the compiler will resolve it when we initialize a reference to the interface type. Then we start implementing the concrete class. There will be a constructor which receives a sequence. Even if the sequence is, in fact, a list, this class will make a copy to avoid affecting the data structure held by its caller. But now we meet the first technical issue, the one which we have already encountered before. Shall we sort the list here or create a lazy sorted list as before. This is the first time that we are constructing a true collection, and true collections come with their native costs. Therefore, that general rule that constructors should not do extensive work might not hold any more if we chose to view it that way. It is reasonable to populate the collection when the constructor receives data, sustaining all associated costs, if for no other reason than because all .NET BCL collections are operating in that way. This step ensures that collection will work as a snapshot of

its input data. Caller might change the input data right after calling this constructor, but that would not affect this collection instance simply because it has already made a copy of the original data. We could ask for a comparer as another constructor argument and sort the list right away. I will also trim excess memory allocated by the list because there will be no more items to add to it. Leaving excess space is pure waste in that situation. With the sorted list in our hands, the other methods will be next to trivial to implement. We will delegate item selection to the underlying list and account, too, and get a numerator, of course. That was easy, as we delegated all work to the encapsulated list. The only member that remains and the only that will ask for engagement is that `GetRange` method, implementing it the right away will take time and investment in design. I will skip that work for now and take a shortcut by copying the section of the list. `GetRange` method on the `List<T>` class is returning a new `List<T>` class, which only contains elements from the specified range. This method of isolating a section of a list is wasteful because it is duplicating all elements in that range, and there will be another round of copying on every subsequent call to the `GetRange`. That is wasteful and, as I said, we will not solve that problem right now. The topic of this module is generics, and we will only produce a bare minimum of functionality to make the demo operational, that includes ensuring that the index is within range of the list. I will also modify `Count` to ensure that it never goes past the end of the list, also asserting that it is never negative. This will give us a valid ordered section of the list, which should become the content of the new fully-sorted list. There are two problems with this approach. First, constructor will copy this list once again, and that would be doubly wasteful, knowing that we have already paid the cost of copying the data. Second, I would have to remember the comparer, so to pass it to the constructor to sort the list again, that would be triply wasteful because the list section is already sorted. You will want to preserve resources when working with collections. One common approach to avoiding inefficiencies is to introduce a private constructor which gives you a shortcut to an efficient construction process of a new object. This class will guarantee correctness of arguments passed to private constructors, with constructors guaranteeing the most time and space efficient initialization of a new object. With the concrete generic class done, we can turn to the main method to try it out.

Demonstrating Concrete Generic Class

I might be happy with what we have accomplished so far and start using this class. But as you will see in just a minute, that will quickly become unpleasant, with code becoming clogged with angle brackets and type names. In the main method, I will prepare the input data, that will be a sequence of Worker objects. We can now prepare a reference to an interface type, `IOrderList`, specialized to Workers. And we will assign it to a new instance of the concrete class, `FullySortedList` of Workers. I'm sorting the data using `RateComparer` defined on the Worker class. We can then print the data out the usual way. Prior to running the application, please remember that this interface is covariant. We can declare the variable as an `IOrderList` of employees as well, Compiler has no objections. Employee is the base type to Worker. Interface is covariant, therefore, the interface of the Employee is a base type to the interface of Worker. If a method requested an interface of Employee, we would be fine to supply it with a concrete `FullySortedList` of Workers. The method would notice nothing strange. Let's quickly run the application so to make sure that the concrete generic class is working correctly. Everything looks in place. If you observe closely, you will notice that payrates of these workers are indeed appearing sorted in the ascending order. The class is operational, but I have an objection regarding code readability. Look at the place where we are instantiating the sorted list. We are making a call on the class with generic type parameter. Now consider a class with two or three type parameters, that would be ugly. What if type parameters had long names, that would be exceptionally hard to manage. And what if I told you that you can avoid specifying generic type parameters entirely, not like this, obviously. But again, I tell you that we can achieve this only not by calling the classes constructor.

Applying Generic Type Inference

The next goal is to avoid specifying generic type parameters if possible, a process known in C# as generic type inference. It only applies to methods and delegates, not to types nor to constructors. To make this line compile, I will apply one nice coding trick which moves type parameters from the type, `FullySortedList`, to a method where the compiler can infer it. I will add a new variant of the `FullySortedList` class, this time, the one which is not generic. I'm adding it to the same source file where the generic class is because their names are the same. Now we can write a static factory function, which instantiates the generic class above. At this stage, observe the generic type parameter `T` has migrated from the level of a class to the level of a method. From now on, it will be subject to generic type inference. In the factory function body, it only takes to call the generic classes

constructor. This is the coding maneuver to move the generic type parameter from a class to a method so that generic type inferences can apply. And there is one more benefit from this approach. You can turn a static factory function into an extension method so that the caller can either invoke it as a static function or to apply it to an object. When we call the extension method, it's containing classes name will not appear at the calling site, losing part of the information about what this method is doing. You should name extension methods more specifically than common static methods. And now we can turn back to the main method where we have identified readability issues. Instead of calling the generic constructor, you can apply a generic extension method to the sequence. This call is equivalent to instantiating the class via its constructor, with one distinction, when the compiler encounters a call to a generic method, it tries to match argument types with the type parameter. Since raw data is a generic sequence of Worker objects and IEnumerable of Worker is covariant, type argument can either be Worker or any type deriving from a Worker. We call this entailing process generic type inference and the C# compiler is routinely applying it. It allows you to remove explicit type parameter from the call and rely on the compiler's logic to figure it out for you. But the process does not end there. Compiler will consider all arguments before making a conclusion. The second argument is this comparer of type IComparer<Worker>. That interface is contravariant. Type parameter must either be a Worker itself or its super type. There are two opposing forces here, one argument asking for Worker or less derived, and another for worker or more derived type. The only type that satisfies both constraints is the Worker class alone, and so the compiler will make a clean decision that generic type parameter is Worker. It will usually be easier for the compiler to infer type parameters than in this example. The overall result is that we have been spared the burden of using angle brackets that undisputed champion of arguments in C#. Even more, common type inference works fine with generic types, too. If I substituted keyword var for both type declarations, code would still work fine. This is the generic code I like to see. The stretch of code, which is generic to the tiniest bit, but with no angle brackets used. The best generic method out there is the one which requires no type parameters. Remember that. And there is one more benefit from having extension methods beyond the generic type inference, they allow for method chaining. Look here. We can start from the input data sequence, then turn it to a fully sorted list with a call to the extension method, and finally format data into a grid and print them out. Still not a trace of generics, but it is generics all the way down. No trace of collections, but there are two of them hiding behind the curtains, one list sorted using the custom comparer here and

another list which helped exercise efficient row and column wise traversals over here, and yet none of those is visible externally. Remember the rule, the best collection type is the one you do not depend on. One more topic remains before we close this module, adding constraints to generic type parameters.

Using Constrained Generics

This is not where improvements in generic methods end. After turning to the static generic factory, we can make one more step that will make it even easier to use. First, if there is nothing specific in the generic class's public interface, then static factories can return the interface. This step is still optional. On most occasions, you will return a concrete class knowing that we can always assign an object to a reference to its parent interface. And now we are ready for the next feature supported by generics, and the last one which we will demonstrate in this module. What if we wanted to support consumers that do not have a comparer? Worker class already exposes a comparable property, hourly rate. So why not let the caller specify that we should sort the list by that property? There is no such an extension method currently, so we need to pay another visit to the static, non-generic, FullySortedList class. Here is the idea. Expose another static factory function, which receives a keySelector function rather than a comparer. One problem here is that comparer requires another generic type parameter, the type of the comparison key, and so we shall add the second type parameter to the methods signature. And then comes the critical piece, this key type cannot be just anything, we want to use it to compare instances of type T. That is where another concept comes in. We can constrain a generic type parameter so to guarantee that it is of a certain type or derives from it. To constrain a type parameter, begin with the keyword where right after the meta declaration, followed by the type parameter TKey here, and then derivation symbol colon, followed by the base type that must be satisfied, IComparable< TKey> here. This declaration says that any concrete type used in place of TKey must be a subtype of IComparable<TKey>. If there are more constraints to specify, separate them with commas. For instance, this says that TKey must also be equitable. In this demo, I Comparable<T> will be the only constraint we require. If you remember, when I was talking about marker interfaces, those that declare no methods, I mentioned that we can apply them to indicate a semantic intention of a type rather than its syntactical quality, like saying that the list is sorted or that a type is representing a database key. This interface is specifying that semantical quality, a piece of knowledge about a class,

but which the class text does not state. Should the method above insist on only operating on comparable objects that are persistent keys of objects it processes, we would attach the marker interface like a badge to the Key objects type. Our domain model does not require this constraint. You get the picture. With the generic constraints specified, we know that keySelector will transform the input object into a comparable object, which we can use to compare items to each other while sorting. I will prepare a helper method that turns the keySelector into a proper comparer. This generic method will also come with two type parameters. We will also constrain the TKey type to implementing IComparable<TKey> so that when the time comes to truly compare keys extracted using the keySelector function, those objects will indeed possess the CompareTo method. Here, you can see the purpose of constrained generics. Without the constraint, the compiler would not be able to confirm that the object returned by the function will have the CompareTo method defined. Look, if I removed the constraint, the compiler would issue an error because now it cannot infer the existence of the CompareTo method. Let me put the constraint back and complete the factory method above. This implementation will delegate to the existing factory function with a single type parameter. Please remember this coding technique, which allows you to reduce the number of type parameters by wrapping one of them into an object that hides it away. This is an important tool, which you should use to adapt one generic type to the definition of another generic type. An important change happened in the main method. The call has immediately switched to an overload with two generic type parameters, but both were inferred from arguments, one to Worker from the sequence of Workers and another to Money from the lambda's return type. Therefore, this is the call to the to FullySortedList of Worker and Money. Generic type inference has saved our grace again, and we did not have to specify type parameters. I will run the application one last time. You will see that workers will still be sorted correctly, only this time using their hourly rate as the sorting key. This completes the coverage of generics in C#. Before we move on, let's briefly summarize what we have learned in this module.

Summary

In this module, we have continued the story of generic types and methods. We have walked through an exercise of implementing a concrete generic type, which has quickly led us to the question of generic type inference. You have seen that a compiler is able to infer concrete generic type parameters in methods. It can do the same in delegates, but not in types and, consequently, they are

constructors. Through careful design of generic methods, you can remove all traces of generics in the consuming code, thanks to the compiler's ability to infer concrete type parameters from methods arguments. In the last demonstration, you have learned to constrain the type parameters and why that is important when developing generics with prerequisites. With generic constraints, you can request a type parameter to derive from a certain type. These constraints will allow the compiler to assert the existence of certain members on objects used in implementation and, hence, to compile your code. From this point on, we shall not talk about the generics per se, but we will be using them for all demonstrations that follow. In the next module, we will turn our attention to associative collections, which we have been evading successfully so far. That will be the opportunity to develop some truly magnificent classes, which are, not the least important, useful in business applications.

Using Associative Collections

Introducing Associative Collections

More than a half of the course is behind us, and here we are about to mention associative collections for the first time. How come? Contrary to linear collections, those that associate keys to objects have a narrow purpose. There must be the key to object mapping, and, not so obvious, one key can only map to one value, otherwise, there would be a collision between objects with the same key. We normally need a reason to use an associative collection, like a performance reason. Most associative collections are fast in retrieving an object by its key and slow in retrieving objects by other criteria such as position. That is directly opposite to lists where retrieval by position is instant and retrieval by other attributes is often proportional to list's length. Cost of searching, inserting, and deleting from an associative collection depends on its underlying data structure. Hash table will yield fantastic $O(1)$ time and balanced tree $O(\log n)$ time on a collection of size n . Compare that to a list where appending an item would cost $O(1)$ time, but both searching and deleting would cost $O(n)$ time. It is the performance of these specific operations the reason to use an associative collection. Hash table and balanced tree are the workhorses of such C# collections as `HashSet`, `Dictionary`, `SortedDictionary`, and the like. In this module, we will walk through a series of requests that will ask for the use of a `Dictionary`, a

SortedDictionary, an ImmutableSortedDictionary, and, eventually, a HashSet. Let's step into code where the first request is awaiting our attention. This is the Money class with a couple of additions. We are defining Money with Amount, Currency, and, particularly important, decimal Precision. Money is still supporting arithmetic operations, only a bit more engaged than before. Adding two money amounts remains possible within the same currency and same precision. This private method named Compatible will throw when the argument violates these conditions. Due to the currency restriction, I have renamed this method to AddStrict. We can only add amounts of the same currency. Precisions must also match, but that is a technical request. Matching currencies is a substantial request. But what if we wanted to put different currencies together? This new method called Add lifts the request for equal currencies, but it cannot return another Money instance. It will return what I call the MoneyBag instead. That is the new model class, its methods not implemented yet, which mixes amounts in different currencies together. It is possible to construct the MoneyBag from multiple Money objects. We can add a money amount in any currency or subtract one so long as the bag contains enough of that currency to subtract. We can reduce a MoneyBag to a single Money instance, but only if all it contains is one currency. Methods preserve the right to throw in case of a constraint violation, like attempting to reduce to Money a MoneyBag containing amounts in more than one currency. Can you implement this class? What would you need to combine amounts of different currencies in the first place? We must store multiple incompatible Money instances. We could hold them in a linear collection such as a List, but not all these objects are independent. Some Money objects will share the same currency, and calculating balances per currency would require a full list traversal. Why not aggregate Money objects with equal currencies? That is where associative collections come to the table, like a Dictionary which can map Currency into Money with that currency. Please observe an intrinsic connection between the key and the object. It would be a defect to store a Money object under a currency different from its own. We must encapsulate such coupling into a class which will ensure that coupling is observed on all instances. That is the reason the outer caller is only passing Money objects to this class, and this class is responsible to index every Money object under its own Currency. Working with collections does not abolish you from applying common principles of object-oriented design. One set of rules is addressing naming. My personal preference is to name associative collections by what they map into what. This one is mapping currency into balance. One last note before we start implementing behavior. This class is encapsulating a collection to a greater

degree of secrecy. It could collapse a million money instances into a single entry if only they shared the same currency. That sounds promising already.

Implementing a Dictionary-based Class

Populating the collection will not be straightforward here. There is a convenient LINQ operator, `ToDictionary`. One overload only receives a lambda, which selects the key, so to index each item by the selected key. The problem with this approach is that `ToDictionary` will throw if it encounters a repeated key because it does not know how to reconcile duplicates. But we do know we add `Money` objects. To cut the long story short, I will simply instantiate the dictionary and then add each item separately, using the instance level `Add` method, which knows how to aggregate money. Next question, is this bag of money worth zero? Maybe. That will be so if the collection of money contains no entries. I will be careful to never insert a zero amount into the dictionary. That was easy. But adding money will be a serious business. To add a new amount, we need to find the current balance for its currency. But that currency might not exist in the dictionary, indexer will outline the problem. This trivial implementation will throw if the dictionary does not contain the requested key. We might try to return a zero amount on negative search, but that would not be possible. `Money` constructor requires precision, and we do not have one here. Throwing the exception looks right in the indexer, but not in the `Add` method. I will define a private search method to work around the problem. It will return an existing item or a zero amount with requested precision on negative lookup. This helper method is of critical importance to the `Add` method, which cannot assume that the argument's currency is known to this object. Add the new amount either to the existing balance or to zero, and we're done. But do not forget to give up if the argument itself is zero, otherwise we would risk creating a zero amount in the dictionary violating this class's invariant. The invariant is a claim that is always true. For instance, the `IsZero` property above is critically dependent on a guarantee that no zero amounts will be there in the dictionary, and so this object must observe that invariant ensuring to never violate it. Look how neatly a complex feature is building on smaller ones. There's a whole lot of code hiding behind these calls, including multiple validations that may throw exceptions. Subtracting will operate in a similar fashion, only with an additional requirement. If the resulting balance is zero, then we must remove its currency from the bag. Please note that the `Subtract` method may throw if the balance is smaller than the amount to subtract. There is a helper method which assesses whether it is possible to subtract the

amount from the existing balance. Before we finish developing this class, I wanted to draw your attention to one bad coding practice here. I am repeatedly indexing balance by currency. That is verbose and open to mistakes. It is usually it is better to encapsulate operations on collections. This single method is now ensuring the objects invariants. One invariant is that every balance stands indexed under its own currency. The other invariant is that any balance in the dictionary will be strictly positive. On top of that, encapsulating these invariants in a method is making the rest of the code more compact and readable, and less prone to errors. Please note that the Remove method will only return false if the specified currency could not be found. We are safe from exceptions. We're getting close to the bottom of this class, converting a MoneyBag to a single Money instance will be easy. I'm using the Single operator from LINQ, which will throw an exception if the collection of values contains more than one element or if it is empty. For callers who are not certain whether it is safe to convert a MoneyBag to a Money instance, there is this Boolean method below returning true when the dictionary consists of a single entry. This completes all functional elements in the class. GetEnumerator will delegate to the dictionary's Values property, which I am conveniently exposing for a helper property getter, also named Values. This is the entire class that aggregates Money objects by their currencies. The most important part is the dictionary with accompanying methods that operate on its content. And now, at last, join me for a demonstration. I will instantiate an empty MoneyBag first, now I need realistic Money objects. There is a convenient static property which will prepare them for me. I want to take 10,000 instances, that is a lot of money. In the end, let me print the object out. If I run the application, you will notice that even though there were 10,000 objects put into the bag, it has managed to aggregate them into only four distinct Money instances. Please observe that every currency has its own precision, that is the ability of MoneyBag to manage precision on a per-currency basis. This completes the demonstration. We have already cracked major design problems. In the next demonstration, I will show you improvements to this implementation.

Indexing Items by a Sorted Key

I have modified the main method to demonstrate an annoying deficiency in this design. Here, I'm instantiating an array of MoneyBag's and then I'm adding money to them in round-robin style before printout. And look at the output, all bags are formatting their content in arbitrary order, that makes it harder to read the data. It is the ToString method responsible this time. It is simply appending the

values in order of their appearance. The same problem would appear in a more serious feature because the dictionary makes no promises about order of items in the value sequence. It makes certain promises about speed, delivered by the underlying data structure. Imposing a sort order might jeopardize performance goals. But what if sort order was a request? Customers are already asking about having currency sorted, and we will use the ToString method to practice. We will print the bag's content out in ascending order of currency codes. Please note that currency is already implementing the IComparable of Currency generic interface. We can sort currencies ascending by their code, which is aligned with the new request. MoneyBag is already indexing money by currency. With comparable currencies at hand, the task becomes trivial. Instead of the dictionary, we can use the SortedDictionary. It will manage sorting by the key for us. There is nothing to fix in the constructor because it is using the target typed new expression. SortedDictionary has the same API as the common dictionary, only with an added promise to sort its content by the key. But sorting comes with a cost of its own, and so Add, Remove, and Search operations on the sorted dictionary or execute in logarithmic time. As its size grows larger, the SortedDictionary becomes slower than the common dictionary. As a comparison, common dictionary relies on a hash table, with $O(1)$ asymptotic time to Add, Remove, and Find an item. Lucky for us, this class is aggregated input objects. The size of the dictionary sorted or not will remain small, irrespective of the number of money items we added to the MoneyBag. Run the application again and enjoy nice and tidy output, with all currencies so beautifully aligned. This was easy. We have changed the way our class operates by only changing the underlying collection. But the next request will not be easy at all.

Wrapping a Collection within an Immutable Class

You have already seen that this class's public interface is giving out no information of a collection it keeps inside. That is great, honestly. But then its public interface is mutable. What is the difference between MoneyBag's Add method and debt of the Money class? All arithmetic operations on Money are immutable, always returning a resulting object, Money or MoneyBag. I want that same ability here in the MoneyBag. Instead of a void, I want all arithmetic operations on MoneyBag to return a new immutable MoneyBag. This demo will be an experiment. Suppose that there is a requirement to implement an immutable model. Such requests are quite common these days, so, this is the opportunity to practice them. Solving the challenge will put stress on the cost of creating a new

instance. Back in the Money class, creating a new instance takes constant time because the number of fields in the new object is constant. It is entirely different in the MoneyBag class where one of the fields is a collection. Constructing a new instance would require instantiating, but also populating, a copy of the dictionary. That increases the method's complexity. It used to cost constant time when a common Dictionary was in use. It grew up to logarithmic time when we introduced the SortedDictionary. Now, with immutability as a requirement, the cost of creating a full copy of a SortedDictionary suddenly becomes log-linear. Well, that is the problem, you see. The Add method operates on a single instance. It makes a little sense to make its cost proportional to the size of the whole collection or worse. We would be fine with $O(n)$ logarithm and time for populating the whole collection though. We had that with a SortedDictionary already. In the case of the MoneyBag class, the number of currencies can grow as large as 300. That is the number of currencies in existence worldwide. If we use the mutable SortedDictionary to model an immutable MoneyBag, our code might run up to 300 times slower than necessary. That is where immutable collections come to the table. Like a SortedDictionary, there is the ImmutableSortedDictionary generic class. You will need to include the System.Collections.Immutable namespace. This will cause changes in the way the class operates. Immutable collections are instantiated using the static factory property, conventionally called Empty. But the greater difference is that calling the Add method, here in the constructor, will construct a new MoneyBag, and changes will not reflect on this instance. When designing nontrivial immutable classes, you can follow that coding pattern of immutable collections. First, make the constructor private, and let it receive the entire immutable state of the new object. That is a common practice. That will make static factory property the only way of instantiating an immutable MoneyBag. Speaking of which, let me rename it to Empty to follow the convention. Factory will now pass that Empty ImmutableSortedDictionary to the private constructor. We move down to the operational part, which is all about this Set method. Remove method, on an immutable collection, returns a new collection instance so that all prior references remain intact. That is the trait of immutable collections. Remove will also work fine when it cannot find the key. It would simply return the same unchanged instance then. And we should set a key-value pair in an immutable dictionary through its SetItem method, which also returns a new dictionary instance. Please note that both of these methods are executing logarithmic time, which is the second-best option when it comes to performance. It is mutable collections that offer best performance. Remember that. You will only use their immutable variants

when there is a request to design an immutable class which encapsulates a collection. And now that both calls are returning a new `ImmutableSortedDictionary`, we can turn this entire method into an expression which evaluates to that new dictionary. This is the immutable design in its core. There are no mutations anywhere inside of the object. Arithmetic methods above are calculating a new balance, but they must wrap `Set` method's result into a new `MoneyBag` instance. Let me help them with another private method. And now we can complete the immutable operations on the `MoneyBag`. Add method will wrap the new balance into a new instance it returns. It compiles fine now. The `Subtract` method will follow suit. Mind the modified return type. And that completes the design of a deeply immutable `MoneyBag` class, which is still encapsulating a `SortedDictionary`, an immutable one this time. Consumers will have to adapt. Where `Money` used to instantiate a `MoneyBag`, it will now have to follow a more engaged protocol, which traverses our three distinct instances of `MoneyBag`. Only the last one will remain as the returned object. We have fixed this one. The main method remains, and then we will run it one more time. Instantiate Empty `MoneyBags`, then fix the loop to let the new versions substitute prior ones. That is how you would normally use immutable objects. We are ready to run the immutable demo. You can witness that nothing has changed in the output. Immutable `MoneyBag` with an `ImmutableSortedDictionary` buried deep inside has just produced the same output as that mutable design based on common `SortedDictionary` did. With this, we have completed comparison of dictionaries, common `Dictionary`, then `SortedDictionary`, and then `ImmutableSortedDictionary`. We will not talk about immutable collections any further in this course. There are better courses in the library explaining functional and immutable design, including immutable collections. Before this module ends, I wanted to show you one more principle of association, which remained uncovered, the `HashSet` collection. In the next demonstration, you will see one interesting application of a `HashSet`.

Identifying Memory Pressure Problems

In the next demo, I'll show you how to use a `HashSet` to reduce the memory footprint of an application by sharing objects. `HashSet` differs from dictionary, in that it uses the object itself as the key. Here is one source of memory waste in our demo model. `Currency` struct is wrapping around the string, ensuring type safety and invariance. This struct is type safe preventing consumers from sending plain strings around when they mean currencies. `Currency` is also ensuring invariants that its international

code will always be non-empty and in uppercase. But how many distinct strings will there be wrapped into these structs? That depends on the context. Usually, there will be no upper bounds to the number of currency codes coexisting simultaneously in a single program run. That is not a problem per se. We are working with millions of objects routinely. But currencies, I mean, what the millions of currencies? International standard ISO-4217 defines less than 300 currency codes worldwide. In a million many instances, the most popular currency codes would repeat 100,000 times each, what a waste of memory. Why don't we share the same string between all the equal currency instances? Sharing an immutable currency will not cause any adverse effects, but it will save memory. And there is more. Frequently shared objects are more likely to remain present in fast CPU cache. Contrary to that, holding numerous copies of the same string will degrade performance of the CPU cache and slow down the entire application. In the next demonstration, we will measure time and memory performance combined in method which operates on millions of currency instances. We will then use an associative collection, HashSet, to improve CPU and memory consumption. We will start off constructing a benchmark, which analyzes CPU and memory performance when allocating the currency struct. There will be a parameter which tells total instances count, and there will be a sequence of currency codes represented by strings. We will incorporate these strings into currency instances later. But this task will not be that straightforward. I want only 10 currency codes to repeat in random order. I will take those 10 from this handy static property called TestCurrencies. And there is one special extension method in this testing library, which I have called RepeatRandomly, which repeats its input objects endlessly and randomly, like a music playlist shuffler would do. Fun fact, this extension method is utilizing the shuffler we developed earlier in this course. Then comes the critical part. I could extract codes here using LINQ, sorry, and claim the feature done. But this would be a terrible mistake, leading to a clearly incorrect conclusion from the benchmark. Keep your eye on this issue because it is sometimes hard to figure out. If we assume that RepeatRandomly repeats instances from the input, then the same instances will appear multiple times in the output sequence. Then the resulting sequence of strings will also consist of only 10 distinct string references repeated over and over with no end. That is not what we need here. We need a sequence with no references repeated. It is the content of strings that repeats causing multiple allocations of equal strings. That is precisely what would happen if you materialized those strings from the database, for example. I have already implemented this protocol forcing a new string instance for every currency object in another extension

method called Codes. This is a correct procedure to prepare test data for a benchmark. Remember it. Do not make silly mistakes in performance testing. And now comes the first Benchmark method, the Baseline. It will simulate creating currencies with no optimizations applied. To cut the long story short, a PlainCreate procedure will populate an array. It will then take the enumerator from the sequence of codes and step to the for loop. In the loop, take the next code and construct a currency instance for the array entry. When done, return the array. This is the entire baseline test. It will cause enormous duplication of strings, wasting memory, and impeding the CPU in its progress through data. Now we need another solution to compare the results.

Improving Sharing of Immutable Objects

Here is the idea how to keep the burden of repeated equal objects in check. That is an old problem partly ignored nowadays due to abundance in resources. The flyweight design pattern explains this very problem and the solution to it. It applies to shareable objects, such as immutable objects, when we notice that equal objects are repeating to such a degree that they start hurting performance. One way out is to store every unique instance into a collection and then substitute it for every equal instance requested later. In this demo, we will continue dealing with repeated strings. You'll remember that the string class is exposing the static method, Interned, for a similar purpose. It is the runtime that manages a collection of Interned strings. This code is constructing two distinct string objects. And yet, after Interning, all three variables will reference the same object. Interning a mechanism has drawbacks. It is global and it is static. And once Interned, the string remains in the global string pool for the rest of the program's lifetime, occupying memory long after any need for it has ended it. In this demo, you will learn how to design your own pool of strings or a pool of any other objects, the lifetime of which would be in your hands. Let's start another Benchmark, the one which will cache known unique objects. Everything is the same as in the first benchmark, so far. But now, comes the change. Since the Currency class is a record, it has the Value type quality implemented already. That allows me to remember unique instances in a HashSet using currencies as their own keys. I will look for the new object in the HashSet and, if found, reject just instantiated currency in favor of the prior one. What a waste of time to construct that first object you must think. But wait just for a minute longer, and you will see the situation turning into an advantage. Negative case remains. When we find a unique currency object, we shall add it to the index for future use. Whichever object survived, that will be the

next element in the array. And now come the questions, which of the two implementations will run faster and which will take less memory? The latter question is easy to answer. This second implementation will only retain references to a handful of unique currency objects, which is much less than in the first implementation. But the question about speed is a tricky one. Look at this method. It is doing every everything the first one does and more, iterating through the array, advancing for strings, and allocating a new currency object. All this code exists in the second method, too, with the overhead, searching the HashSet, and occasionally inserting an element into it. Looking at operations alone, the first implementation should be a clean winner. But on the other hand, being efficient with memory has its merits, too, primarily through an efficient use of the CPU cache. Before I run Benchmarks, I wanted to show you how you can profile memory using BenchmarkDotNet. At a class level, add the MemoryDiagnoser attribute. Benchmark runner will attach a special memory profiler, giving insight into memory patterns exhibited by methods. I will lead you straight to the results. The Ratio column shows that for small collections, plain creation is a better option. But creating 1 million items is about 20% faster. That is not great and you will rarely be pressed to apply caching for such a small benefit. The last column shows memory allocated during the run, but not the memory that remained allocated after the run. BenchmarkDotNet cannot measure that for us, and we will do that in the next module. But the table is still uncovering many long-living objects in the first Benchmark compared to the second one. It is the high numbers of long-living objects that cause concern because, beyond a certain point, they start causing degradation in the CPU cache efficiency. Still, the results are not convincing, yet, that is because we have not been using the shared objects so far. Let me show you what will happen if we try to use the currency objects created by one method or the other. This is the new Benchmark class. All experiments will run on 1 million currencies, but we will pass through them 1, 3, or 10 times. The experiment will sum up the lengths of all currency codes. That operation will require access to all objects, and I hope to see shared objects serve their content faster. There are two Benchmarks, one using currencies created by the PlainCreate method and the other using those coming from the caching method. I will show you the Benchmark results. Here, you can see that the running time of the cached solution is improving with growing number of passes through the data. The more you access the shared objects, the faster your code will be. Absolute times are also important, with many objects and repeated access to each times grow larger as well, ending up in a loss of worth 40 milliseconds. I measured that on high-end hardware. On a commodity hardware, lost time will grow

to hundreds of milliseconds. And even 40 milliseconds can be a lot, that is the ping time across the Atlantic Ocean and back. Lesson learned from this measurement is that we should share immutable and other shareable objects when a lack of sharing causes the degraded performance.

Summary

In this module, we have put stress on associative collections. We have applied a dictionary to collect and aggregate objects that share a common key. Even without aggregation, you would use the dictionary to assert and quick search for items by their key. Dictionary is an extremely efficient collection, executing all common operations, adding, searching, and removing in constant time. Past that simple example, you have learned that SortedDictionary is offering efficient manipulation of items sorted by their key. All operations, adding, searching, and removing run in logarithmic time, which is acceptable in most practical applications. After that, you have learned the practical technique of turning a class which encapsulates a collection into a deeply immutable class. By encapsulating an immutable collection, ImmutableSortedDictionary, in this example, you can make your class safe for multithreading or in any design based on immutable models. In the last demo, we have analyzed memory usage patterns in a program with millions of objects. You have learned that failing to share references to objects causes lower CPU efficiency and higher memory footprint. We have proven by measurement that the simplest object caching solution based on HashSet can nearly double the speed of a piece of code and reduce memory consumption to a small, constant amount. That completes practical coverage of associative collections. In the next module, we will focus on constructing classes around the association principle. We will first implement that transparent cache of equal objects and then turn to another problem that will ask for specialized knowledge and skills to solve.

Engineering Solutions Using Associative Collections

Favoring Simple Design over the Complex One

Hello, and welcome back to the course on collections and generics in C#. In this module, we will design a class built on the principles discussed in the previous module, and then we will construct classes that build on top of the class that builds on top of collections. Before these module's end, you will see how collections are used in solving a real-applicative problem. In the previous module, we have shown that a lower memory footprint can lead to faster execution. You have seen that HashSet can help accomplish that goal. But the problem was that we had to create an object before testing if it is a duplicate. Why not remember constructor arguments and produce an earlier object right away when the same argument values appear? That is where we can apply a Dictionary to map constructor parameters into a fully constructed object so to avoid constructing it again. Of course, I will develop that solution in the form of another Benchmark method. My plan is to compare its performance to the two solutions we have developed earlier. This method's name comes from functional programming, where memorization is a technique of associating arguments to the result it produced on an earlier call. When the same arguments repeat, the Memorized method produces a known result without executing the method's body. Method must have no side effects and its result must solely depend on argument values for memorization to be valid. Both conditions hold in the currency constructor. I'm storing Currency codes, strings, in the Dictionary, mapping them to corresponding Currency instances. But the Currency class constructor is turning codes to uppercase so that they all look uniform. That is why the Dictionary must use a special comparer that comes packed together with the Currency class. It repeats string comparison logic from the currency. When the time comes to construct Currency objects, we will consult the Dictionary first. Only in case of a miss, shall we construct a new instance from the code, and then add it to the Dictionary for future reference. This pattern is known as the read-through cache. We are reading through the cache. In case of a miss, the cache will forward the call to the ultimate data source, a constructor, in this example. Here are the results. This begins to look like a real thing. Object construction time is down by 40%. What agitates me, though, is the complexity of the solution based on the Dictionary. In the first demonstration in this module, we will implement a solution based on the HashSet, which is simple and straightforward. We shall return to the solution that uses a Dictionary in the second demonstration. I'm owing you the measurement of total remaining memory in these free implementations. We need to measure objects that remain after the test and we must measure that to ourselves. This private static method will receive a function that performs one specific Benchmark, producing an array of currencies. You can use the KeepAlive method on the

garbage collector to hold a reference so that the object is not subject to garbage collection before this line, that is until we complete the measurement. This method is measuring a single solution, so I will have to call it three times to complete the memory Benchmark. You will see what we knew already, that both caching solutions will only cost us the size of the array itself, plus a few bytes more. One-million currencies will cost 8 million bytes to hold references to strings on a 64-bit machine. We cannot avoid that expense. But the non-cached solution has and another 32 million bytes on top of that to store repeated strings. That is a huge waste of memory, which hurts even more when we remember that in this application, it is certain that the array mostly consists of duplicated objects. In the next demonstration, we will construct a universal caching class based on HashSet.

Implementing a Transparent Object Cache

This is the empty generic class which will utilize the idea of using a HashSet to store shareable objects of type T or its subtypes. But this will be a constrained generic class. Target type will implement the IEquatable interface so that we can discover equal objects. There will be a method which substitutes a cached version for an input object. Benchmark is giving us a head start because we have already implemented this feature there in a form of a working prototype. By the virtue of copy and paste coding style, this class will hold a private HashSet. In the method's body, we ask the HashSet for a known replica. If there is none, we shall remember the argument and turn that one into the result. That completes the class that implements the basic principle of sharing immutable objects. In less than a dozen lines of code, we can make another piece of code twice as fast with its memory footprint turned to bytes. But our work on the design does not end here. There are consumers to attend. A consumer would use an instance of this cache. But once that cache goes out of scope, all unreferenced cached objects will be subject to garbage collection. What other scenarios could we support? Why not expose another method which maps multiple objects in one call? This is one useful abstraction, which can even remove the entire cache out of the caller's site. Let me show you. This is the static class with extension methods we used in previous modules. I will add one more static method to apply the TransparentCache to a sequence of objects. The caching object will jump in, process the sequence, and fade into oblivion. But a sequence it returns will be as compact as possible, due to sharing of objects it contains. And even this is not where my appetites for caching end. What if I wanted to cache objects of all types in imaginable, so long as they implement the IEquatable of their own type? Here is

the idea. I want to have another TransparentCache, this time not generic. This class will expose that same caching method only this time having its own generic type parameter with the same constraint as before. And now comes a puzzle for you. How would you implement this method? The trick is in supporting all kinds of types T, which we cannot guess ahead of time. Even worse, there will be no common base for those types other than the object class. Now think, if you want to give this problem a try, this is your opportunity before the beginning of the next clip.

Designing Classes Based on Association

Please consider this development a demonstration of working with generic types. I will turn this non-generic cache into a cache of caches, given a type, get me a TransparentCache specialized to that type. Unfortunately, C# does not allow generic properties. Properties look too much like data, and one can only add generic behavior to the class, that means generic methods. Now comes the substance. How do we store caches of diverse types here? I will repeat that idea with using a Dictionary to index cached objects, this time mapping a type into a cache of objects of that type. But there is no common type for all those caches, I will have to work around the strong typing constraint and dump all those incompatible objects onto a single pile of the Type object. However, this is not just any pile. The Dictionary is structuring this pile around Type objects used as keys. That is the virtue of dictionaries, they add a semantic meaning to values they store. Look how neatly we shall reinstitute the strong types back again. Ask for a weakly-typed object index under the requested type. Then apply type test and set pattern to turn it safely into a TransparentCache specialized at type T, whichever it is. This completes the positive case. In case of a miss, we shall create a brand new cache of type T, remember it for future use, and return it to the caller. This is the coding maneuver which is playing on the edge of the object substitution principle, and yet the caller will know nothing of the drama unfolding behind this tame, simple method we're about to implement. Its body will do nothing more than delegate to the contained single typed cache. Finally, it will cost us a little to include that same multi-item caching method as above. Only this time, I want to show you what a LINQ variant of the same operation would look like. It will amuse you to see that we can implement the entire method above in a single call to the Select extension method from LINQ. And the last addition is to define another extension method which would remove the requirement that the consumer must own a cache instance. I want to offer those two caching methods, one dealing with a single object and another with

a sequence of objects. This time, I want to show you one practical trick. Look at the first solution we had. Calling this method for the second time would cause creation of a new set of unique objects. In this extension class, I will apply a different protocol. There will be a single, static Cache reused across calls to extension methods. But is this safe? It is common in .NET to make static members thread safe. Lucky for us, we do not need true multi-threaded sharing of caches. In C#, you can declare state as a static field and then apply the ThreadStatic attribute to that field. There will be one static cache instantiated per thread, making that cache thread safe. Should someone call static methods from multiple threads, performance would degrade gracefully, but never to become worse than that of the first operator, which used to instantiate new cache per call anyway. By the way, do you remember what I said about a global string pool maintained by the intern static method on string? I complained that interned strings will mingle around forever. This static solution would behave the same. However, in this design here, you have an option to instantiate a cache yourself, and to manage its lifetime in your best interest. We can now complete the extension methods. Those two will be the simplest of all methods. Please observe how generic typing inference has jumped in so we do not have to specify the type parameter in method calls. That is why I will usually opt for a generic method on a non-generic class, rather than the other way around. Let me quickly show you how we can use these classes. I will repeat constructing a sequence of 1 million currency codes with frequent repetitions. I will apply the cached operator and collect objects into a new list, and this is the number of bytes added by the second list. What shall this code display? We have seen these results before. The first list is appending additional memory per-list entry due to repeated strings with equal content. The TransparentCaching component is working as expected, mostly taking space for the list itself.

Compensating Service Access Time in Chatty Protocols

We will now move to a more complicated example. You will remember the Worker class from previous modules and its base class employee. Those classes used to model people employed in the company. There will be a new model in this demo, the WorkerItem class, which will model a chunk of work assigned to a single worker. And there will be two additional interfaces in the domain model representing services, possibly remote ones, that we can use to fetch workers and work items. In the next demo, we shall use these two services to really fetch some objects. But, as you will witness, that experiment will not pass without troubles. Code is telling more than words. There is new code in the

Worker class and its base Employee. Workers now have a GUID, which uniquely identifies every person modeled by an object. And there is a new model here, a class which represents work done by a worker with specific ID charged this much. I have also added two services, one is managing Workers, while the other is managing WorkItems. This abstract service will provide access to WorkItems. But if we got hold of a WorkItem and wanted to inspect the Worker associated with it, that would require accessing another service to search by a GUID. And there the problems begin because this service might take time to respond. I have prepared a demonstration how these services work together. The WorkItems service is producing a steady stream of items, we're taking 1000 of them, and then, for each item, we're making one call into the Worker's service. We would say that communication between these two services is chatty, consisting of many small requests. This is important from a theoretical and practical point of view, where we observe two metrics, time to first byte and time to last byte. Time to first byte is the interval after which the first piece of response arrives back. We assume that the data will keep flowing so that the entire response arrives at time to last byte. Time to first byte then, the volume of data in a time interval depends on the network, distance, the underlying hardware, and other external conditions, which we cannot affect from a running software. What we can affect, though, is coarseness of communication between services. Coarse-grained requests, also called chunky, would ask for a large quantity of data in one request. Time to first byte will then amortize across a long sequence of bytes, eventually making transfer rate the dominant metric. Contrary to that, asking for a small amount of data makes the entire communication cost equal time to first byte. Keep repeating the message, and you will have that dreaded chatty communication protocol, where the sum of delays waiting for the first bytes will dwarf all other delays. Back to code, this parameter defines time to first byte. Every call to find, the method will take five milliseconds. Not much is it? There is a joke saying that speed of light is not a config parameter. Five milliseconds equal round trip time across 750 kilometers distance. If your software depends on a truly remote service, then be ready for delays much longer than five milliseconds. Let's contact this remote service 1000 times, then, five seconds total time, plus a tiny little bit more. My guess is that tiny little bit is the actual work. Those five seconds, that is the time we wasted, the cost of chatty communication. One way to address this problem is to chunk requests. Unfortunately, that depends on the remote services design, which we might not be able to affect. The second best then is to make chatty calls less frequent. In many systems, you can retain responses for a period and still

consider them up to date. In this demonstration, I will show you how you can implement the so-called read through cache. The consuming service would make a request to the cache which hides the producing service, only forwarding the request if it knew no prior response from an equal request. Cache will then remember the response on its way back for future use. In object-oriented design, it is common to implement cache as a decorator around the service, implementing the same interface as the original service. Deciding whether to use cache or not then boils down to choosing one object composition or the other. In the following demo, we shall implement a general purpose read-through cache that will help improve performance of demo services.

Designing a General-purpose Read-through Cache

Every cache comes with three defining elements, definition of items it contains, indexing method, and purging policy. I will define indexing and content by adding generic type parameters to this class. Generic constraint will ensure the TKey type is a qualified search key. The purge policy is defining conditions to remove an item. Without purging, cache would grow indefinitely, possibly bringing the process down. One of the simplest purge policies used in practice is referred to as the least recently used, LRU, that explains the name of this class. When capacity is at the limit, the cache will remove the least-recently-used item. You can implement LRU policy by holding a linear collection sorted by the last access time. Whenever you touch an item, move it to the end. The item that appears at the head of the list is nothing but the least-recently-used one and the first candidate for expulsion. The problem, however, is in finding the item in the middle of the list. It is, therefore, common to add an index built on top of the linear collection, which gives us quick access from a key to the location in the list. Why don't we only hold the index then? That is because the linear collections remembers the time ordering of items, which a dictionary cannot do. The last problem to solve is migration of items. We can address that problem by using a LinkedList rather than a common list based on an array. The LinkedList is promising $O(1)$ time to add, remove, or refresh an item, irrespective of its position. The index built on top of the LinkedList can be a common dictionary, which also guarantees $O(1)$ time for all three operations. This is one of the finest pieces of design in object-oriented programming. Try to analyze it at your leisure. This is the blueprint of a custom LRU cache, which we will implement in C#. Cache will keep record of its capacity, as a maximum number of items to hold before activating the purge policy. I will also prepare the ground to support changing capacity during the lifetime of a cache. You will see

that feature later. There will be that LinkedList of items, the least-recently-used item coming first. And there will be the Dictionary, which holds actual items indexed by their associated keys. But now, look here, it is the index of ListNodes we need. This dictionary is indexing the list, not the items. But during purging, we will need the backward link, too, from an item to its key, so to remove the Dictionary entry, too. Every ListNode will hold both key and a value together. Quite verbose, right? But I am sure that you can see this data structure in motion already. And when you do it up in your head, you will notice that one vital element is missing. Finally, the read-through nature pops up. This cache will also require a strategy to fetch an item from an external source, in case of a miss. Constructor will take care of capacity and strategy. Collections will start off empty. The stage is set, and we can define a method that looks for an item by the key. Reading from the cache has a positive and a negative branch, a.k.a., cache hit and cache miss. This one is modeling a hit. After obtaining the node, we can return the value it holds. LinkedListNode exposes property for its contained value, a tuple here, from which we will poke our own value component, a tad confusing. But it is too early to return yet, we need to reinstitute the LRU order, remove the item, then add the node's content to the very end of the list, making it the most recently used item. At last, we'll return the newly created LinkedListNode, which will substitute the existing index entry. And now we are done. Cache miss will be easier to implement. We must ask the external source for the value because we do not have one, then put a key-value tuple into the LRU list, making sure to also add the node to the index. But one operation is missing, adding an item to the cache increases its size, and we must activate the purge policy to ensure that we have not breached the capacity. This method will ensure a valid size of the cache. While number of item succeeds sees the capacity, the extract key of the least recently used item, remove the head element from the list, and remove a corresponding index entry. This is the least-recently-used purge policy. Simple and effective. Above, in the cache miss case, we need to run the purge policy before completing. I also promised the ability to change the capacity after initialization. With the purge method already implemented, that will be simple. Set to the new capacity and force purging. This completes the LRU cache implementation. In this demonstration, you have learned how two fundamentally different collections, a LinkedList and a Dictionary, can operate together on the problem of indexing the same set of items by two distinct criteria, access time and access key. In the next demo, we will apply this cache to improve performance of a service with the chatty interface.

Improving Performance of a Chatty Service

It would be fair to say that this cache assumes that items it contains should remain valid forever. It is common in practice to let the user specify time to live, TTL, as another parameter, so that cache invalidates items as they grow old. Nevertheless, in the course dedicated to collections and generics, I tried not to drift too far away into the vast fields of object-oriented design. I wanted to demonstrate this beautiful interplay of collections. This is the crux of the read-through cache, and now we want to build that same capability into a service, which implements this interface. In object-oriented design, we can add a feature to a class by defining a decorator, another class implementing that same interface, so to tap into the calls to the original class and modify its behavior. This is a special `WorkersService` implementation. It will intercept calls to another `Worker` service. It also contains a cache of workers, where `fetch` strategy is defined the method of the encapsulated service. Now we stepped down to implement the interfaces members. Getting all items will only delegate to the contained service, but magic will unfold when it comes to finding a worker by ID. There, this service will effectively call the cache rather than the encapsulated service. The cache itself might end up calling the other service, but only after a miss, and that is all. We could set up a local service if we wanted, which remembers responses provided by the remote one. Now we need a practical method to wrap a service into the caching decorator, that is where default interface methods of C# 8 can really shine. You can define a full-blown method directly on the interface including its body, like the one which returns a cached version of this object. If you're worried about an interface depending on a concrete class, then you can opt to define this method close to the concrete class as an extension method on this interface. You have seen me doing that already in earlier modules. Every implementation will now inherit this method without having to provide its body. I will use the opportunity to show you one valuable optimization trick. If this service is already the caching one and, specifically, one with capacity which is at least as large as requested, then this object is already satisfying the request. Only if this condition is not satisfied, wrap this instance into a new caching service. Default interface methods are convenient way to extend an object model without disrupting existing classes. For example, Java is implementing its streams library, which corresponds to a LINQ in C# via default interface methods defined on collection interfaces that existed long before streams. Nevertheless, development is over, and we are ready to demonstrate the power of the new model. I will let the cache capacity range from 0 to 10, which is the total number of distinct workers in the system. Zero capacity means no caching. Inside the loop, there

will be a cached variant of the WorkerService, which will get us the Worker objects. What can we expect from this solution? This is the moment of truth. The first row, that without caching, will appear after five seconds again. But as we keep repeating the experiment with ever a larger cache capacity, execution time will drop rapidly. Thanks to the high repetition rate of equal requests, read through cache is helping draw execution time close to zero. There is one hidden detail in this printout. In practice and especially for data shaped by human behavior, there will be an uneven distribution with certain items appearing more often than others. I have simulated one such distribution of workers, where some workers are more active than the others. The result is that even a small cache capacity has cut execution time severely. Further, capacity growth leads to diminishing returns, as cache is now keeping even those items that never accumulated to large delays anyway. With this successful demonstration, we shall close the module and engineering principles of classes based on associative collections. Let's briefly summarize what we have learned.

Summary

In this module, you have learned how to utilize principles of object-oriented design to construct classes that depend on associative collections. In the first experiment, we have applied the HashSet to reduce a group of equitable objects to only a handful of unique instances. That has caused significantly lower memory footprint, plus a measurable significant cut in execution time. However, the stress was on constructing classes that encapsulate collections. And in that light, you have seen the design which conceals HashSet as a private component. In the second experiment, we have constructed a generic LRU cache and applied it to keep record of known objects that would otherwise be costly to create or fetch again. We have then engineered an object model in which a read-through LRU cache is but a component inside the service decorator, making caching a declarative feature. In both designs, caching the objects became a matter of configuration, that is the success of object-oriented design, where collections were only the toll. On top of that, you have once again witnessed that the best collection is the one that the caller cannot see. We have used the opportunity to discuss communication protocols design, showing how chattiness may cause a distributed system to grind to a halt. A local cache, if applicable, can reduce the number of calls made to a slow service and improve overall software performance. It looks like I was only using dictionaries to cache objects so far. And, in fact, its greatest value is in quick look up, running in $O(1)$ time on top of its $O(1)$ one time and space

climate to add an item. No linear collection can compete with a Dictionary in that arena. If you thought that I was over using the word cache in this module, then try to look at dictionaries from this angle. It is their unique ability to seek an item by the key in constant time, the most prominent reason to use dictionaries in practical coding, and what is that but caching. Try to view your own designs based on dictionaries in that way, as caching the items for fast retrieval at a later time. We shall continue in that spirit. In the next module, we will return to designs based on linear collections, only this time with a lot of added work around them.

Engineering Custom Linear Collections

Seeing the Power of Linear Collections

Welcome back to the course on collections and generics in C#. In the previous module, we have started building practical solutions that rely on custom classes that are wrapping collections. That is the usual course of action when building complex domain models. In this module, we will continue with the same effort, this time, using linear collections in the bottommost layer. I find great value in linear collections, as you may have already suspected. A list can have a structure beyond its physical organization, the array. One such structure is the sort order. We use the lists with sorted content in previous modules, that is very useful in practice where we often encounter the request to work with sorted data. But that is not the only possible attribute of the underlying data. For example, we could sort a list topologically. Imagine a model which is generating documents or files. A single document could depend on a handful of other documents. To complete working with one document, we must complete all the documents it depends on. Any linear order where a dependent document appears to the right of its predecessors is a valid topological sort order. There can be many distinct topological sort orders each as valid as any other. Even though the word sort appears in the name of topological sorting algorithms, they usually have little or nothing in common with comparison-based sorting algorithms, such as the quicksort. There are other data patterns supported by lists and you will encounter them in practice. A list can appear as the result of querying the database or from

deserializing data. It can be a source for subsequent serialization for a network response, for instance. That is what makes a list the data structure of choice in numerous algorithms. To further exercise the power of lists, we shall build on top of the order the list abstraction we developed in earlier modules. In the first demo, we will design a class which calculates data percentiles. That is a common request, in practice. In the second demo, we shall implement a class which removes extreme values from the list, the so-called outliers. Let's begin with the first requirement. Given a sequence of objects as sorting criterion and the percentage value, design a solution which returns an object such that the exactly the requested percentage of objects are less or equal than the returned object in the sort order. For example, tell us the pay rate limit for the lowest paid 10%, then the lowest paid 20%, and so on, of all the workers in the company.

Implementing a Percentiles Calculator

Here is the new interface which models the `IPercentilesReader`. Please note that it is a covariant interface because all it does is to return items from a sequence. What sequence, you may ask? Well, that is someone else's problem, as in the interface segregation principle. This interface will only model the reading end of the percentiles. Other interfaces, such as the `IEnumerable` or `ReadOnlyList`, or that one of ours, `IOrderedList`, will cover the input segment, that which defines the items from which we are selecting the percentiles. You will see why this separation of interfaces is so important. The `GetPercentile` method receives a relative position of the requested item from the interval zero to one, excluding one, because that would fall just outside the collection. For example, value 0.1 would correspond to the 10th percentile, and so we want to implement this interface. A concrete class would have to impose a sort order and then to select an item at the requested position. Those are two pieces of functionality, and we do not like to implement two features in one class. That is where separation of concerns jumps in. Do you remember the `IOrderedList` interface? That interface is modeling a list with its content sorted using some sorting criterion. The interface has no clue what criterion, that is someone else's concern, again. It is the separation of concerns all the way down, you see, and it works for us. Look here, I'm introducing a concrete class named `OrderedPercentilesReader`. It will implement the `IPercentilesReader` interface, that explains the `PercentilesReader` part of this class's name. Now comes the ordered part. The object will depend on another object, which implements the `IOrderedList` interface. This class is applying object composition.

I will now show you why that is important. If you remember, we had this class named `FullySortedList`, which implements the `IOrderedList` interface. It applies full sorting to the underlying list collection in $O(n \log n)$ time, and then guarantees subsequent access in $O(1)$ time by the item's position within the sort order. The problem with this list was in-depth a log linear initial cost that we must pay before we can enjoy the merits of its fast access later, that would also be the cost of constructing a percentile reader as well because this constructor would have to wait for the `FullySortedList` heavyweight constructor to complete first prior to executing this trivial assignment. And for what reason, may I ask? For the reason of poking a few items out and ignoring myriads of others. Now you remember that horror story of implementing a custom partial quicksort algorithm, which will only sort segments of the underlying list which we are effectively accessing. Why not pass another implementation, some future `PartiallySortedList` object, into the `items` field? That would allow the `GetPercentile` method to force sorting of specific points inside the list while leaving the rest of it unsorted. My guess is that such a marvelous solution would spare lots of resources on a larger list. Now think of this, if this class opted to inherit from the `FullySortedList` rather than to accept it as a component, then that design would close the door for using a more optimal `PartiallySortedList` the day it becomes available. I have said this several times and I will repeat again, solving one set of problems, like dealing with a sorted collection, does not abolish you from applying all other principles of object-oriented design. Favor object composition over class inheritance, that is a decades-old principle that will save your grace every single time you remember to apply it in complex domain modeling. Now that I have completed the design phase, which took several minutes of my time, the coding phase will take mere seconds. Given the percentile, which must fit into the interval zero for one, get me the valid index of the requested element in the list. I will even improve code readability by using named arguments for literals zero and one, which will help understand that the validation extension method assumes an exclusive lower bound and an exclusive upper bound. This will help the reader understand that the result returned from this method will always be a valid index inside the list. Long story short, this is the entire percentiles reader that operates on an existing `OrderedList`.

Demonstrating the Calculator

Time for a demonstration. I want to use this class to calculate the distribution of salaries among our workers, but I fear that using this class will require a lot of typing on the consumer's end. Let me add a

few bits to the design with the sole purpose of improving the caller's experience. The first step, define a convenient record type which wraps a percentile position and value together. These two values naturally fit together, anyway. The second step, make the `GetPercentile` method return the `Percentile` record, not just the list item. That would give that item a proper meaning, helping avoid bugs due to not understanding what the object at-hand represents. Please note that the `Percentile` class is both accepting and returning an of type `T`. I had to remove the `out` keyword from the interface to support this new type. The interface has just become invariant. I made a corresponding change to the concrete `PercentilesReader`, too. It is producing the `Percentile` object now. But a more substantial improvement is awaiting us. I want to include a couple of extension methods that would help the caller obtain percentiles without having to bother with object composition. This is that `Operators` class where we used to add similar fluent operators in the past modules. I would like to `GetPercentiles` from an `OrderedList`. This extension method will apply to an `OrderedList`, and it will also ask for the first `Percentile` requested, and the step to apply repeatedly to fetch all the others. I am sure that this extension method will simplify the consuming code. And I have another bright idea in mind, what if the caller wanted to see the extreme values as well, the 0th and 100th percentile? Here is the extension method which will prepend, then append, the extremes to the sequence of percentiles produced by the extension method above. Mind the guard clauses. We do not want to push duplicates through the output, nor do we want to cause an exception. How do you like these utilities? Let's try them out in the main method. I will get the workers, 1 million of them. Nothing can stop me because I have an excellent sorting algorithm of the `List` class under the hood. Now I have the `IOrderedList` object at hand, get the 10th, 20th percentile, and so on, including 0th and 100th. Print them out. I cannot wait to see this code in action. After a moment of waiting, the table will be there. You can see that 20% of all workers receive around \$73 per hour or less, for example. That is the kind of an analysis that customers will do and the reason they asked for this feature in the first place. Still, observe that it took three lines of code and two seconds of CPU time to come up with this aggregated analytic for a million people.

Implementing the Outliers Remover

We will now move to the next requirement. Customers will analyze data, like this hourly rate curve we pulled out in the previous demonstration. Customers often wish to remove items that lie at the ends of

the curve. Those items often do not follow the same patterns observed closer to the middle and, hence, may lead to slightly incorrect conclusions. For instance, we may note an abrupt rise in the direct-most 10% of the curve. If our customer wanted to know the upper bound of how much the workers charge for their services, my guess is that they would rather say it is \$130 per hour, not over \$150. We call these data points at the tears of the curve, outliers. They lie outside the main trunk, and it is a common practice to remove them from the data before the analysis. As an example, you may refer to earlier modules where we use the BenchmarkDotNet intensively. If you run any CPU Benchmark again and read the details of its console output, you will surely find that the Benchmark runner is also removing outliers before constructing the table with final measurements. The removal of outliers is an essential operation in financial analysis, hence the request. Given the sequence of objects and the sorting criterion remove the specified percentage of items that lie at the ends of the sort order. Straight to code, here is the interface that defines a method according to the requirements. It is the interface segregation principle at work, once again. Given a value in the interval zero for one, this interface promises to return a sequence with that relative part of some other sequence omitted. For instance, if we passed 0.2, the bottommost and the topmost 10% of the other sequence will be omitted from the output. Another entity will deal with the sequence and with the sorting order, each item in code will focus on one task and solve it so that no other piece of code will need to deal with it. This is the entity which answers the question, what sequence? And this is the interface that answers the question, what sort order? We have seen all this already. The public method will subdue the argument to validation and turn it into the absolute count of items to remove. And this is the method that will do the work. The poor-man's solution would be to calculate the starting index, then the exclusiveEnd index, which considers the possibility of an odd count value, and to iterate through the `OrderedList` to serve its items out. That is the code one would write if they never heard of code reuse. I bet all this code would already exist in any implementation of the `IOrderedList` interface. Here, the `OrderedList` exposes the `Range` method returning another `OrderedList`, which implements the `IReadOnlyList` interface, which, in turn, inherits from `IEnumerable`. Voila. The `GetRange` method returns `IEnumerable`. We happily stepped back to the consuming class ready to replace this inflated implementation with a simple call to an existing method. Please observe that we are now calculating arguments the way `GetRange` expects them. Anyway, this looks great. Are we done now? The `OrderedList` will take the burden of procuring the middle range, right? That is how it ought to be. Let's

take a look. This is the `GetRange` implementation and the only concrete class that implements `IOrderedList`, the `FullySortedList` class. What it does is to construct a new, `FullySortedList` object and pass it the result of the native list `GetRange` method. Well, bummer, this inner `GetRange` method will construct an entire new `List` object and copy the items into it in the outliers remover that will always be close to the entire list, like 80 or 90% of it. I told you this will happen. If you remember, in the earlier module where I had cut the corner to implement this `GetRange` method, I told you that I will have to pay another visit before the end. Why couldn't I just leave the method this way and call it done? It is because of the callers. The poor caller must choose between implementing a for loop, which is bad, and calling an existing method, which will do that in, I don't know, about five times the time, which is even worse. It is our duty, as programmers, to always construct optimal solutions in public methods because we cannot guess the context that will utilize our classes. The time has come to implement this method the right way.

Designing a Projection onto a Collection

The `FullySortedList` class is wrapping around a common list, holding it in a so-called frozen state. When the list is instantiated, it is subdued to a few mutating transformations, filling the items in, sorting, and trimming. But once those initial mutations end, the data structure will remain intact for the rest of its parent object's lifetime. By extension, the `FullySortedList` object will remain immutable once constructed. You can build on this reasoning and construct a so-called projection on top of the entire object. Instead of constructing the new object and paying the associated costs, you can create a thin wrapper around this object, and only recalculate the indices when accessing this same list. Let me show you. This class is a thin wrapper around an `OrderedList`, any `OrderedList`. The cost of its constructor is next to nothing. But this class will also implement the `IOrderedList` interface. It will only grant access to a part of the contained `OrderList`, though, outlined by two parameters, an offset and a count. This is the offset inside the contained list, and this is the maximum count of successive elements this `OrderedList` will see. Please note that its own indices will stretch from zero through `Count - 1`, as usual. Looking from the outside, this object is just another `OrderedList`. But on the inside, this class will maintain an offset by which is translated across the contained list, as well as the `MaximumCount` of items it can serve. What will the actual count be? That will depend on the contained lists current count, and we will calculate its value with every request. We shall sanitize the input and

the initialized state in the constructor. And now comes the substance. The indexer will sanitize the index and then offset it before passing it to the underlying list. That is the crux of the projecting list. All it does is to safely translate its own zero-based indices into a range of indices belonging to the other list. Please note that this implementation does not require the immutable `OrderedList`. However, abrupt changes to the contained `OrderedList` while an operation is in progress could cause an `IndexOutOfRangeException`. The immutable design, like one applied to the `FullySortedList` class, dismisses that threat natively. The `GetEnumerator` method should be straightforward as well. I will prepare a helper property getter, which calculates the upper bound for an iteration. The method itself will be straightforward, a simple loop. This is the entire thin wrapper around an `OrderedList`, such that we can obtain access to a window, a section of the `OrderedList`, such that that section remains ordered and applicable to all scenarios where in `IOrderedList` is requested. And now comes the prize, the projection's own `GetRange` method implementation. How would you implement this method? Think carefully because this method is the place to demonstrate full understanding of the concept of a projecting class. Should I instantiate another projection on top of this object? That would certainly solve the problem, wouldn't it? Yes, it would, of course. But at what cost? Someone would call the indexer on the result. The indexer would recalculate the index and apply it to the private `OrderedList`. But the private list is also a projecting one, that list indexer would also translate the index before applying it to its own private `OrderedList`, and again, and again, all until the call reaches a true `FullySortedList`. Why in directions if we already possess a non-projecting `OrderedList` right here? To demonstrate real understanding of action projections, I will construct a new projection on top of the same `OrderedList`. That would require mapping the arguments as well, the new index is this projection's `Offset`, plus the requested index. And the new count will be bounded by this projection's maximum extent, not by the underlying `OrderedList`. This expression is quite verbose. Please take your time to figure it out. Pause the video if you must. It all boils down to combining the new transform with the transform applied by this projection to obtain a final transform to apply to the original `OrderedList`. The caller would see nothing of this mess. All they will see is a nice object implementing the `IOrderedList` interface and nothing more. Only we shall know that that object is a thin wrapper, and I do mean thin, around a `FullySortedList` object, which is, in turn, a thin wrapper itself around the plain list. The `FullySortedList` class will now wrap itself into a `Projection` object, ensuring that consumers will remain safe from any performance penalties. Our precious outliers remover will apply the most

straightforward solution, knowing that that solution will be the fastest and cheapest of them all. We are now ready to demonstrate its capabilities.

Demonstrating the Outliers Remover

I have added an extension method to remove outliers from an ordered list. There is almost nothing to see here. This method will make it easy to build fluent expressions that incorporate outliers removal along other transforms. Here is the demo code that isolates and then sorts a million workers, a perfect place for the new operator. Please remove 10% of outliers from either side of the pay rate's curve. Thank you. And another convenient operator, which I have prepared just before this demonstration, the one that will extract the range of the sequence, isolating the minimum and the maximum value according to a custom criterion. I will print the salary range out, but also the time it took to calculate it. This last item will be important later when we start combining the operations. I am running the application, and after a period, you will see that reduced range of hourly rates which excludes the extreme values. And this is the time it took to calculate the result. We shall refer to this duration later. This experiment was a success, and as any success in programming, it has caused our beloved customers' appetites to grow larger. They want to see the pay rate percentiles after we remove 20% of outliers. This is precisely what they want. Sorted the list, remove outliers, and then read the percentiles. And it does not compile. Why? Here is the GetPercentiles operator. It requires an ordered list. When chained after the RemoveOutliers operator, it will only receive an IEnumerable. That is why the call to GetPercentiles will not compile, But why is IEnumerable the result here? It is because the OrderedOutliersRemover returned it that way, even though it did have an ordered list in its hands. But why would it do that? It is because its parent interface, the IOutliersRemover said so. Here it is. This interface cannot depend on the IOrderedList interface. That would be wrong. What if some concrete operation does not sort the data and we fall victim to impedance mismatch between an abstract interface and a concrete class. I will now show you a coding maneuver that will resolve this unpleasant situation. In the concrete type, operate on concrete types the best you can. Take all the power you can get, but that will cause a compile-time error because now the class does not implement the interface's method. No problem. Give it first-class attention by providing an explicit interface implementation. IOrderedList is a subtype of the IEnumerable, and hence, this method can safely delegate to the more concrete method below. The class compiles fine, and we can think of its consumers now. Those that

only have a reference to the interface will have to satisfy their needs with only the abstract method, and those with more specific requirements will have to give up the abstraction and talk to a concrete type. There, their call to the method with the same signature will end up calling a more concrete variant with a more concrete return type. Like the RemoveOutliers operator already does, it is calling into the concrete class, and this call will return the ordered list. There are no obstacles to return that same type further. Back to the main method at last where you will witness that the code is compiling fine. And there is more than just compiling the source in this. When we run the application, it will print the data points, but a more important detail to note is that the entire operation, both removing the outliers and calculating percentiles, has completed in about the same time it used to take to only remove the outliers. That is because the outliers remover is only returning a projection to an already sorted list. It does nothing that consumes time, nor does the percentiles calculator do anything more than picking a handful of items from a list of enormous size. All this time you see here is only the result of sorting the list at the very beginning. All subsequent operations cost the next to zero, thanks to our beautifully composable object model. This was the last demonstration I planned for this module. Let's briefly summarize what we have learned so far.

Summary

In this module, we have continued to design production-grade classes that are building on abstract concepts developed earlier during the course. We have proven once again that collections should remain deep beneath the surface and far away from the domain-related code. In the first demonstration, we have developed a class which calculates percentiles of a sequence of objects. This is a common requirement, especially in financial portions of code. In the next demonstration, we have dealt with the problem of eliminated outliers from a sequence given the sorting criterion. We have built both solutions on top of the abstract concept of a sorted collection. That has proven invaluable at the very end where we have demonstrated a capability to compose the two solutions, percentiles calculator and outliers remover, in such a way that neither of them will spend any additional memory or CPU when executing in succession. Bottom line is that the general-purpose operations on collections naturally belong to the infrastructural part of a larger application. We should implement them optimally and design them with composition in mind. Upper layers, those that speak in terms of a business domain, will rely on composability to build complex and powerful features, never paying attention to

any collections that will in fact execute those features on a bare metal. We will continue the pursuit of ideal designs in the next module. We will mention queues and stacks for the first time. Those are the last remaining collection types to cover in this course.

Engineering Queuing Solutions

Identifying the Role of Queues and Stacks

Welcome to the module in which we shall mention queues and stacks for the first time since this course has started. These are also linear collections, but they differ from common list, like collections in operations they expose. The generic Queue class in .NET is exposing Enqueue and Dequeue operations with $O(1)$ time and memory complexity. Stack allows Push and Pop, also with $O(1)$ time and the memory bounds. You will hear terms FIFO, first-in, first out, for the queue and the LIFO, last-in, first-out for the stack. If you ever encountered a FIFO-looking problem, then the Queue class will be your friend. If your problem exhibits a lethal nature, then there is the Stack class to use. FIFO problems are comparatively rare in practice. But LIFO, let me put it this way, one does not expect to come to work and write code for a LIFO kind of a problem on any given day. You will normally use queues and stacks as a temporary storage, a parking lot where objects abide until conditions are satisfied for them to leave. Queues are quite common in multi-threaded code, where they're used to decouple producers from processors working on separate threads. In this module, I will give you a few ideas where queues and stacks may come handy in common, single-threaded code. Queues are useful in a large group of problems known as the windowing problems. Imagine just counting items issued in the last hour or summing them up. We call this section of a sequence, a window, a sliding window that moves across objects. There are quite a few useful algorithms based on the sliding-window technique. Their power lies in the ability to aggregate items. When we add an item to its right, the window would attribute it to the accumulated value. When the item goes out of scope to the left, the window would remove it from the accumulated value. The power of sliding window algorithms is in implementing the aggregation function that runs in $O(1)$ time. We will apply a couple of

algorithms based on this technique. Spoiler alert, sliding windows look like queues, and we will indeed use a queue to implement them. But there will also be a stack hiding deep inside one of them. Stay with me, and you will see one of the most marvelous queueing algorithms you will ever see. We will start with a simple requirement. A company is issuing work items during the day, each with cost assignment time. At every moment through the day, report the number and total cost of work items assigned in the last one hour. More requirements will come later. This is the demo code which will only display the work items. `FetchWorkItems` will serve a sequence of objects describing work. I'm printing the first 100 items to show you the data. Here they are. Now we must aggregate work items by counting them and summing their costs up.

Implementing the Sliding Window Class

Here is the class which will implement the sliding window process, on top of which we will design more specific classes. The sliding window expects the outer caller to add items to its tail end and to remove them from the head end later. That will make the window slide across the sequence of objects of type `T`. We might also supply the window with a predicate that will make it slide automatically. The return value would tell whether to remove the head or not, given a freshly-appended tail. The default predicate will always return false. The window will not remove anything unless we instruct it otherwise. The window could also expose some of its state, the Head element, the Tail element, and the Count of elements currently in the window. For those programmers who did not quite understand the purpose of the sliding window, we might also include implementation of the `IEnumerable<T>` interface. The power of the sliding window is in fast segregation, not in iterating through the window, and so we come to the accumulation. I have already defined an interface which represents an object which can accumulate its value during lifetime. You can add an item and then remove it later. The accumulator will then serve the accumulated value quickly, and if you wish, many times over, without spending any additional resources. Please observe that the accumulator is contravariant on the Item type, but covariant on its own accumulated type. This will be useful in deciding window implementation. And here is an implementation which will let you construct any Accumulator you like, providing three items, the seed value and two lambdas, to add and to remove an item from the accumulator, respectively. The accumulator is seeded so that it starts off with a valid value. Its value will reflect any item added or removed. The Value property itself runs in constant time. Total cost of passing a single item for the

window is the sum of costs of the two strategies. The most effective sliding window algorithms have these strategies implemented in constant time. Sliding the window across the entire sequence of size N , that costs $O(N)$ time and $O(N)$ memory. That is as close to optimum as one can get. Now you know why sliding window algorithms are among the most powerful algorithms in programming. The `SlidingWindow` class will accept zero or more accumulators and store them in a private array. This completes the set of public members and components that define an operational sliding window. Now we step inside the class to implement its internals. For one thing, the item's storage. Since we add items and then remove them from the window at disparate moments, we must hold them in a collection. I have already mentioned that the sliding window looks like a queue, and queue items are coming in from the right, and the queue items are leaving to the left. Now we can step to the members to implement them. I will start with these members that only projected the state without actually doing anything with it. The `Head` element delegates to the queue, throwing if the queue is empty. The `Tail` will be more engaged, remembering the last appended element in a private property throwing on its own when needed. And the `Count` property will also delegate to the queue. We have warmed up and now we can step down to implement the two substantial methods. To `Append` a new item, we need to consult the auto-removal predicate first and to remove items from the head of the window, as long as they keep satisfying it. Then, place the item to the end of the window and do not forget to update the last assigned item expected by the `Tail` property getter above. Then, visit each `Accumulator` and inform it of the new item in the window. This is the entire procedure to add an item to the window, now the one that removes an item. I will first `dequeue` the item. This call will throw an exception if the queue is empty. Then inform the accumulators that this item is about to leave the window, then return to the caller. The `GetEnumerator` method will delegate to the queue. Iterating through the queue does not `dequeue` the items, that is important to note. This is the entire general-purpose sliding-window algorithm. It allows extension for the accumulators and auto-removal predicate, which we are adding at run time. In the next demo, we shall specialize this algorithm to report on work items issued in our demo application.

Implementing the Work Items Window

This is the class that models a sliding window for the sequence of work items. The window keeps a record of a specific interval, which defines its maximum extent at any given time. The window will

serve us the information listed in the customer's requirements, namely the total number of work items assigned during the interval and some of their costs. That is what the caller will expect from an object of this class. And now this classes prerequisites, its own demands for the caller to fulfill before producing the responses. The caller must add items in the correct chronological order. And if there is no item on the input, the caller must tell the current time so that the window can expel outdated items. Without the latter operation, the window could not guarantee correctness of content and total cost results. Expose the Head and Tail element for the caller's evidence. Of course, these properties will throw if access to when the count is zero. This is the entire public interface of the time-sliding window of work items, which counts the work items and sums their costs up. I have demonstrated a common coding practice once again, define the public API consisting of prerequisites and the results the type will promise. Only when that is over and when the design passes the sanity check, step into the members to implement them. Everything beyond this point will be kept secret from outer caller, a.k.a., encapsulated. We know that there will be a sliding window of work items under the hood. It will do the heavy lifting for us, but it also has prerequisites. We must tell it when to remove an item. I will prepare a private method for that and implement it later. The other requirement is the accumulator which it will use to accumulate the costs into a MoneyBag. I'll construct this accumulator first, seed it with the empty bag, then pass two lambdas that pad and subtract money from the MoneyBag. That is the accumulator. When someone asks for the total cost, we shall pass the question to the accumulator. Back to the constructor, we are ready to instantiate the sliding window. Please note that I am passing an Accumulator of WorkItem and the MoneyBag, but a SlidingWindow class is accepting an Accumulator of WorkItem and object. Thanks to the IAccumulator interface being covariant on the second type parameter, this assignment is valid. This new object will then do everything in its power to maintain the window, the structure of work items that span the specified interval of time and not a second longer. And it does have a substantial computing power that it will unleash every time we forward the call to its Append method, for instance. The window will update its internal state, as well as that of the accumulator. It only takes reaching out for the most recent values. We have completed one half of this class, the querying half, now to the other half, the commands the caller will issue. When the caller sets the time, the object will purge the outdated work items, that means to remove the current head while there are items in the window and the head should be removed. I have carefully avoided code repetition. I have implemented the time comparing condition in only one place. Both public

methods, Add and SetCurrentTime, will eventually pay a visit to this line of code here. This completes implementation of the time-sliding window that aggregates work items. There are some 30 lines of code, but only about three lines that are really doing something sensible in this class. The majority of the hard work is delegated to the SlidingWindow class. It is a reusable implementation of the general SlidingWindow algorithm. It is applicable to a wide range of other domain-related classes like this one, a very nice example of code reuse. We are ready for a demonstration.

Demonstrating the Sliding Window Algorithm

Here is the demo code which I have prepared. We shall use the sliding window to aggregate work items starting at 8 a.m. This method Report is defined below. It is printing out the time of the measurement, count of items in the last hour, and their total cost. That is exactly what the customers have requested. This will fetch all the work items from 8 a.m. to 6 p.m. There is also this timeout variable. Since we are operating on time and time is a treacherous dimension, things grow old, even when we're not watching. There will be a simulated timer, which will trigger periodic reporting. Imagine that a head item in the window was assigned at five o'clock, whatever items came later, we will set the timer.trigger inspection at six o'clock precisely because that is the time when that head item must expire. We shall consume work items by advancing through the array. If the timer is set and its time is before the next item is assigned, then this is the logic to execute. Tell the current time to the window and print the report at that time. Otherwise, if the new item has arrived before or at the time of the timer, then just add it to the window and the elect the window purge outdated content as usual. Advance to the next item for the next loop iteration. Of course, print the report, but at the time of the item, which is the new tail. Then comes the logic that will either clear the timer or set it to the expiration time of the current Head element. This code is complex because it is simulating execution of a realistic system driven by time. Here is the report. There are so many numbers printed here, I'll show you the diagram made of these numbers, instead. This diagram shows the total cost of work assigned in the last one hour at any moment. You can imagine business-making decisions, such as assigning more people to support during peak hours or planning a downtime during periods with a low activity. Long story short, customers are delighted with the new features. And guess what, they want more. At every moment in time, they want to know the work item with the highest cost assigned during the last one hour. That is the new request. No problem, we can do that, right? We have the sliding

window algorithm, right? It supports all kinds of accumulators, right? Well, there is no easy way to say this, there is no such accumulator that will give you the maximum of the window in $O(1)$ time. The problem occurs when we remove the current maximum from the accumulator, which item will take its place as the next maximum, the maximum window turns into a search problem, and it has no trivial solution. The sliding window algorithm is fine, though, the accumulators are not. We need something more powerful to solve this genuine problem.

Constructing the Maximum Queue

I will tell you a funny story about implementing a FIFO structure using two LIFO structures. In plain English, I will construct a queue using two stacks, as the items flow in, in repeated Enqueue operations, we push them on to what I will call the input stack. And there will be another one, the output stack. That one will become important when the queue is requested. We cannot just pop the item from the input stack because the first item in the FIFO order is at its bottom. But we can pour the entire input stack over into the output stack, turning the sequence of items on its head. Surprise, surprise, the first item in FIFO order is now the first item in the LIFO order in the output stack. We can dequeue by popping off the output stack, and again for the next the queue, and again, and so on. Any intermittent Enqueue operations would be pointed to the input stack again because we must not spoil the FIFO order established by the output stack. Only when it happens that the subsequent dequeue operation bounces against an empty output stack, shall we repeat that process of pouring the entire input stack over onto the output stack. The process repeats endlessly, accepting all inputs through the input stack, serving all outputs from the output stack, and pouring the content over on every dequeue request when the output stack is empty. What would be the purpose of such a structure? That is hard to tell. Someone may have invented it during the funniest data structure competition, or could it be true that a manager once asked for a queue and developers really didn't like him and wanted to hurt him, so they handed over a double stack. I will now tell you something that will wipe that smile off your face. There can be a gate between the two stacks through which all the items must pass during migration. Why not insert an aggregating component there which would attach information to the items as they pass through? Now you see that this mad joke can turn into a serious design if only we could construct a useful aggregator for the items, like the one that calculates the running maximum. We will now design one such aggregator. We can maintain the maximum of the input stack, updating it every time

a new item is enqueued. Then the repackaging occurs and we face a brand new issue. We know that all items will leave the output stack in their own time. What if an item that we just popped were equal to the current maximum, which of the items below it is the runner up and the one to become the new and undisputed maximum? That we cannot tell. That is where that mysterious maximizer comes to the picture. It will start fresh when the transfer operation begins with no current maximum defined. It will accompany every trespassing item with the maximum attained so far, including that very item. The first item will be its own maximum. All other items will compare to the current maximum and either accept it or supersede it. When the process ends, every item on the stack will record the maximum of all items below it and itself. The item at the top will tell us the current maximum of the output stack. But if someone popped it off in a dequeue operation, that would reveal a new top, and the new top will contain the new maximum, the maximum of all the items that remained on the output stack after the queue. Isn't this a marvelous algorithm? At any moment, the maximum of all items in the queue is the greater of the two-stack maximum. If one is missing, then the other is the winner. And if both are missing, then we shall throw an exception back in the direction of the unsuspecting caller, their fault. This data structure is the so-called maximum queue, and we will now implement it in C# in its full glory.

Implementing the Maximum Queue

This is the skeleton of the `MaximumQueue` class, a generic class which exposes the same set of methods as the common BCL queue does, which implements the `IEnumerable` interface. And on top of all that, does what no other queue can do, exposes the `Maximum` property getter. The `MaximumQueue` requires a comparer. Now to implementing the private parts. There will be the `inputStack` along with a Boolean property, which assesses whether it contains objects. And the `InputStack's` `Maximum`, which throws when the stack is empty. There will be the `OutputStack`, with every item accompanied by the running maximum. There is also a convenient Boolean property, as well as the `calculated` property getter, which isolates the maximum from the topmost output pair or throws if there is none. And now we come to the operations. I will spare myself some typing with a helper method, which chooses a larger out of two items. Now to the essence, the operation that will push an item onto the `InputStack`, initialize or update the maximum, and only after that, push the new item onto the stack. Then comes that complex process of transferring input items to the output stack. I have adopted a habit from functional programming to design decorative methods. This one will ensure

that there are items in the output stack regardless of whether there were any already or not. If the output stack is not empty, then take the shortest way out. Otherwise, if there are no inputs, fail. There is no way to ensure the existence of output items if there are no items to put in. And only now can we start transferring the items, carefully maintaining the running maximum. The first item is its own maximum. The others will have to compete for the throne. This loop will leave the input stack empty, and the output stack non-empty, as promised. To Pop from the output stack, ensure there is output to Pop, then Pop unconditionally. This completes the internal logic of the MaximumQueue data structure, now we need to wire it to the public methods, which I will move up for better class readability. Count will be the sum of counts of the two stacks. Simple. And queuing is to push to the input stack and the queuing is to pop from the output stack. The Peek method will be more engaged. We must ensure there is output first before peeking into it. This is the public interface of a queue. The special thing is about the Maximum property getter, which is the unique trait of this queue implementation. When there is no input, it is the OutputMaximum that decides or throws. Otherwise, if there is input but no output, then the input stack will tell the maximum. Only if both stacks are non-empty, shall we pick the greatest of the two partial maximum. Enjoy it. The MaximumQueue general purpose collection is so fundamentally different from all BCL collections. There is quite a lot of code in its private members. This data structure is objectively complex, but it is implementing an operation which may have seemed unimplementable only 10 minutes ago. This collection has an amortized $O(1)$ complexity for enqueue, dequeue, and getting the maximum operations. It is the queuing that causes troubles because the queuing one item might cause transfer of potentially a large number of items to the output stack. Every item will sustain two push, two pop, and at most, two compare operations. Calculating maximum costs, at most, one comparison. In the next demo, we shall use this queue to implement the customer's request.

Demonstrating the Maximum Queue

We shall now demonstrate the sliding window with extended capabilities. The sliding window algorithm now acknowledges that some aggregators will need the entire queue content to complete their duties. It allows us to inject the queue upon which it will operate. This extension point is optional. Consumers that do not need a special queue can omit the parameter and the sliding window will substitute what I have called a common queue. You will see these details in a second. You can already see where this

is heading. We shall inject the maximum queue for our custom analytics and then read the current maximum from that queue object. There is no abstract queue in the BCL. To make the queues substitutable, I had to define an interface of my own. These are the members of the BCL Queue class including the parent interface, IEnumerable. Some programming languages support mixins, C# does not, so I had to provide a wrapper class to effectively add the IQueue interface to the BCL Queue class. That completes the infrastructure. The MaximumQueue class will also implement the IQueue interface. It will add a new ability to do WorkItemsTimeWindow class. The class will maintain a new component, the MaximumQueue, which extracts the maximum work item using its cost as the goal. We shall pass this queue to the sliding window algorithm. The MaximumQueue will analyze all the work items that pass through the sliding window, apply its magic, and tell the most expensive work item at any moment. This is object composition at its finest. We didn't have to modify any of the algorithms implemented, so far. We didn't risk adding defects to code. We only had to change class constructors to support an extension, even the main method will remain the same. That complex time simulator will remain intact. It is the report we will modify. All modifications are confined in the imaginary presentation layer. If there are any work items in the window, then print the maximum cost out. Otherwise, default to the Empty MoneyBag. We have just solved the most complicated customer's request by only applying a different object composition at run time. That demonstrates the immense power of object-oriented design. And I must emphasize that I was not cheating in this demo nor was I planning for this solution. There is nothing artificial in the design demonstrated here. Starting from customer's requirements, I have applied the principles of collections and the principles of object-oriented design, and this is what I got in the end. There are four columns displaying current time, the number of items, the sum of their costs, and the maximum cost of an individual item, all taken for the last 60 minutes of time. Not the least important, we have generated the entire report in time proportional to the number of work items in the system. Here is the diagram we had before showing the total worth of work items as the day progresses, and here is the corresponding diagram displaying the current maximum. If the customers wanted to see when during the business day the most expensive tickets are being processed, then this is the diagram for them. This completes the last demonstration in this course. Before farewell, let's summarize this module.

Summary

In this module, we have outlined one large category of algorithms in programming which are based on the sliding window technique. You have learned how to use a queue and an accumulator to aggregate objects belonging to a subsequence. There are many real-world tasks that you can solve using this simple idea. Its greatest source of power is in accumulating operations that run in constant time, making it possible to construct a demanding analytical table in time proportional to the number of incoming objects. In the rest of this module, you have learned how to construct a brand new collection, also called the maximum queue. It is interesting to note that the maximum queue is based on cooperation of two stacks. So in the end, you have seen a stack in action as well. If you ever encountered a genuine problem which requires a collection with special properties, then the process demonstrated in the final demo in this module will show you the way. You will probably refer to literature and papers on specialized collections and then implement one of them in your code. There is nothing scary in building your own collections, as you have witnessed in this module. We have arrived at the very end of this course. You have seen all kinds of collections applied to solving practical business-related problems, from playing arrays and lists, over to SortedList and LinkedLists, to dictionaries, sorted and immutable dictionaries, ending with queues and stacks. You have seen that every kind of a collection has its purpose and the problem domain in which it will outperform other collections. And where all collections fail, you will design a collection of your own by applying common principles of programming. Not to forget, we have been using generic types all the way through. We have applied virtually all aspects of generics in C# to accomplish tactical goals in design. I hope you have gained valuable insight on how to apply collections and generics in business-oriented C# code. More than anything else, I hope you will apply the ideas from this course in your everyday work.